

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



TRẦN NGUYỄN ĐỨC MINH

PHÁT TRIỂN CHATBOT TƯ VẤN Y TẾ

**ĐỒ ÁN NGÀNH
NGÀNH CÔNG NGHỆ THÔNG TIN**

TP. HỒ CHÍ MINH, 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



TRẦN NGUYỄN ĐỨC MINH

PHÁT TRIỂN CHATBOT TƯ VẤN Y TẾ

Mã số sinh viên: 2251012095

ĐỒ ÁN NGÀNH
NGÀNH CÔNG NGHỆ THÔNG TIN

Giảng viên hướng dẫn: TS. TRƯƠNG HOÀNG VINH

TP. HỒ CHÍ MINH, 2025

LỜI CẢM ƠN

Đầu tiên, em xin chân thành gửi lời cảm ơn đến Ban Giám hiệu Trường Đại học Mở thành phố Hồ Chí Minh, cùng toàn thể quý thầy cô trong Khoa Công nghệ thông tin đã truyền đạt những kiến thức quý báu, luôn tạo điều kiện thuận lợi cho em trong suốt quá trình học tập và nghiên cứu tại trường.

Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Trương Hoàng Vinh đã tận tình hướng dẫn, đưa ra những định hướng, góp ý quý báu giúp em hoàn thành đồ án này.

Mặc dù đã nỗ lực hết sức, nhưng do vẫn còn hạn chế về kiến thức và kinh nghiệm nên đồ án vẫn không thể tránh khỏi những sai sót. Em rất mong nhận được sự góp ý từ quý thầy cô để có thể hoàn thiện và phát triển kỹ năng của bản thân hơn trong tương lai.

Em xin kính chúc quý thầy cô luôn có thật nhiều sức khỏe và giữ mãi nhiệt huyết để đồng hành cùng sinh viên trên con đường khám phá tri thức của công nghệ.

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TÓM TẮT ĐỒ ÁN NGÀNH

Chatbot tư vấn y tế được thực hiện với mục tiêu xây dựng mô hình hỗ trợ người dùng tra cứu, tham khảo thông tin sức khỏe nhanh chóng, an toàn và thuận tiện. Hệ thống ứng dụng mô hình Retrieval-Augmented Generation (RAG) kết hợp với mô hình ngôn ngữ lớn (LLM) để đưa ra câu trả lời chính xác dựa trên cơ sở tri thức y khoa cung cấp và lưu trữ dưới dạng vector. Dữ liệu được thu thập rồi xử lý từ các định dạng (PDF, CSV), sau đó lưu trữ chúng dưới dạng vector embeddings bằng công cụ ChromaDB kết hợp với mô hình nhúng ngữ nghĩa của HuggingFace. Khi người dùng đặt câu hỏi, mô hình sử dụng cơ chế tìm kiếm ngữ nghĩa để truy xuất các thông tin liên quan, sau đó mô hình ngôn ngữ lớn (LLM) sẽ cho câu trả lời ngắn gọn, dễ hiểu và có tính an toàn. Ngoài ra, mô hình cũng được thiết kế với khả năng trả lời câu hỏi dựa trên bối cảnh của lịch sử hội thoại và mở rộng thêm dữ liệu mới khi cần thiết. Kết quả của dự án này sẽ mang lại một giải pháp hữu ích trong việc hỗ trợ tư vấn y tế cơ bản, giúp người dùng có thêm hiểu biết về thông tin sức khỏe từ câu trả lời do chatbot phản hồi thật nhanh chóng và an toàn. Tuy nhiên, mô hình chỉ đóng vai trò cung cấp gợi ý ban đầu mang tính tham khảo, không thể thay thế các chuyên gia bác sĩ có kinh nghiệm và chuyên môn cao.

MỤC LỤC

LỜI CẢM ƠN	1
NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN.....	2
TÓM TẮT ĐỒ ÁN NGÀNH.....	3
DANH MỤC TỪ VIẾT TẮT	7
DANH MỤC HÌNH VẼ	8
DANH MỤC BẢNG	10
MỞ ĐẦU.....	11
Chương 1. TỔNG QUAN ĐỀ TÀI.....	12
1.1. Giới thiệu về đề tài.....	12
1.2. Lý do chọn đề tài này	12
1.3. Mục tiêu của đề tài	12
1.4. Phương pháp thực hiện	13
1.5. Bố cục đề tài.....	14
Chương 2. CƠ SỞ LÝ THUYẾT	15
2.1. Chatbot AI.....	15
2.1.1. Mô hình ngôn ngữ lớn (Large Language Model – LLM)	16
2.1.2. Retrieval-Augmented Generation.....	18
2.1.3. LangChain.....	20
2.2. Django	23
2.2.1. Giới thiệu Django	23
2.2.2. Kiến trúc Django	23
2.2.3. Triển khai Django	24
2.3. ReactJS	25
2.3.1. Giới thiệu ReactJS	25
2.3.2. Kiến trúc ReactJS	25

2.3.3.	Triển khai ReactJS	28
2.4.	Các công cụ và thư viện hỗ trợ	29
2.4.1.	MySQL Workbench	29
2.4.2.	Git/Github	29
Chương 3.	HỆ THỐNG CHATBOT TƯ VẤN Y TẾ	31
3.1.	Giới thiệu hệ thống.....	31
3.2.	Phân tích hệ thống.....	32
3.2.1.	Lược đồ use case.....	32
3.2.2.	Đặc tả use case.....	33
3.2.3.	Sơ đồ tuần tự.....	41
3.3.	Thiết kế hệ thống.....	49
3.3.1.	Sơ đồ lớp.....	49
3.3.2.	Thiết kế cơ sở dữ liệu	51
3.3.3.	Thiết kế giao diện	54
3.4.	Kiến trúc hệ thống.....	60
3.4.1.	Sơ đồ kiến trúc hệ thống.....	60
3.4.2.	Các thành phần trong kiến trúc.....	61
3.4.3.	Cơ chế hoạt động của kiến trúc	63
3.5.	Kết quả đạt được	63
3.5.1.	Chức năng Đăng nhập	63
3.5.2.	Chức năng Đăng ký	64
3.5.3.	Chức năng Lấy lại mật khẩu.....	64
3.5.4.	Chức năng Trò chuyện, tư vấn	66
3.5.5.	Chức năng Quản lý thông tin cá nhân	67
3.5.6.	Chức năng Đổi mật khẩu.....	68
3.5.7.	Chức năng Quản lý người dùng.....	68

3.5.8. Chức năng Quản lý dữ liệu chatbot	69
Chương 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	72
4.1. Kết luận	72
4.2. Hướng phát triển	72
TÀI LIỆU THAM KHẢO	74
PHỤ LỤC	75

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
AI	Artificial Intelligence
NLP	Natural Language Processing
API	Application Programming Interface
UI	User Interface
Virtual DOM	Virtual Document Object Model
RESTful API	Representational State Transfer API
HNSW	Hierarchical Navigable Small World
RDBMS	Relational Database Management System
ML	Machine Learning
REST	REpresentational State Transfer
JSON	JavaScript Object Notation
XML	eXtensible Markup Language
HTML	Hypertext Markup Language
HTTP	HyperText Transfer Protocol
SQL	Structured Query Language
ERD	Entity Relationship Diagram

DANH MỤC HÌNH VẼ

Hình 2.1. Kiến trúc Transformer của mô hình LLM.....	16
Hình 2.2. Mô hình RAG và các công nghệ phía sau nó	19
Hình 2.3. Kiến trúc của LangChain.....	21
Hình 2.4. Kiến trúc của Django.....	23
Hình 2.5. Kiến trúc của RESTful API.....	24
Hình 2.6. Props và State trong việc quản lý dữ liệu	26
Hình 2.7. Minh họa Lifecycle của một component trong ReactJS.....	27
Hình 2.8. Cách thức hoạt động của ReactJS	28
Hình 2.9. Mô tả Github.....	29
Hình 3.1. Lược đồ use case	32
Hình 3.2. Sơ đồ tuần tự chức năng Đăng nhập.....	41
Hình 3.3. Sơ đồ tuần tự chức năng Đăng ký	42
Hình 3.4. Sơ đồ tuần tự chức năng Lấy lại mật khẩu	43
Hình 3.5. Sơ đồ tuần tự chức năng Quản lý thông tin cá nhân.....	44
Hình 3.6. Sơ đồ tuần tự chức năng Đổi mật khẩu	45
Hình 3.7. Sơ đồ tuần tự chức năng Quản lý dữ liệu chatbot	46
Hình 3.8. Sơ đồ tuần tự chức năng Trò chuyện, tư vấn.....	47
Hình 3.9. Sơ đồ tuần tự chức năng Quản lý người dùng.....	48
Hình 3.10. Sơ đồ lớp.....	49
Hình 3.11. Lược đồ cơ sở dữ liệu của hệ thống	51
Hình 3.12. Giao diện Đăng nhập	54
Hình 3.13. Giao diện Đăng ký.....	55
Hình 3.14. Giao diện kiểm tra Email.....	56
Hình 3.15. Giao diện xác thực OTP	56
Hình 3.16. Giao diện Đổi mật khẩu mới	57
Hình 3.17. Giao diện Trò chuyện, tư vấn.....	57
Hình 3.18. Giao diện Quản lý thông tin cá nhân.....	58
Hình 3.19. Giao diện Đổi mật khẩu.....	59
Hình 3.20. Giao diện Quản lý người dùng	59
Hình 3.21. Giao diện Quản lý dữ liệu chatbot.....	60
Hình 3.22. Sơ đồ kiến trúc hệ thống.....	60

Hình 3.23. Giao diện chức năng đăng nhập	64
Hình 3.24. Giao diện chức năng đăng ký	64
Hình 3.25. Giao diện tìm kiếm tài khoản bằng Email	65
Hình 3.26. Giao diện xác thực mã OTP	65
Hình 3.27. Giao diện đặt lại mật khẩu tài khoản	66
Hình 3.28. Giao diện chức năng trò chuyện, tư vấn	66
Hình 3.29. Giao diện tạo thêm phiên trò chuyện	67
Hình 3.30. Giao diện tìm kiếm phiên trò chuyện	67
Hình 3.31. Giao diện quản lý thông tin cá nhân	68
Hình 3.32. Giao diện thay đổi mật khẩu tài khoản	68
Hình 3.33. Giao diện quản lý tài khoản trong hệ thống	69
Hình 3.34. Giao diện xem chi tiết tài khoản	69
Hình 3.35. Giao diện quản lý dữ liệu học của chatbot	70
Hình 3.36. Giao diện thêm dữ liệu học cho chatbot	70
Hình 3.37. Giao diện xem chi tiết dữ liệu học cho chatbot	71

DANH MỤC BẢNG

Bảng 3.1. Đặc tả use case chức năng Đăng nhập	34
Bảng 3.2. Đặc tả use case chức năng Đăng ký	35
Bảng 3.3. Đặc tả use case chức năng Lấy lại mật khẩu.....	36
Bảng 3.4. Đặc tả chức năng Đăng xuất	37
Bảng 3.5. Đặc tả use case Quản lý thông tin cá nhân.....	37
Bảng 3.6. Đặc tả use case Đổi mật khẩu	38
Bảng 3.7. Đặc tả use case chức năng Quản lý dữ liệu chatbot.....	39
Bảng 3.8. Đặc tả use case chức năng Trò chuyện, tư vấn	40
Bảng 3.9. Đặc tả use case chức năng Quản lý người dùng	40
Bảng 3.10. user	52
Bảng 3.11. chatsession	53
Bảng 3.12. message	53
Bảng 3.13. knowledgebase	53

MỞ ĐẦU

Trong bối cảnh công nghệ thông tin đang phát triển mạnh mẽ, trí tuệ nhân tạo (AI) ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực, trong đó có y tế. Vấn đề về sức khỏe luôn là một mối quan tâm lớn được người dùng tìm kiếm xuyên suốt, tuy nhiên việc tiếp cận với các nguồn tài liệu chính thống và đội ngũ chuyên gia y tế thì không phải lúc nào cũng dễ dàng. Vì vậy, cần đặt ra yêu cầu thiết yếu là có một giải pháp hỗ trợ người dùng tiếp cận thông tin y tế một cách nhanh chóng, an toàn và đáng tin cậy. Nắm bắt nhu cầu đó của người dùng, Chatbot tư vấn y tế được triển khai nhằm xây dựng một mô hình có khả năng tiếp nhận câu hỏi và cung cấp thông tin tham khảo dựa trên cơ sở dữ liệu y khoa đã được chọn lọc giúp người dùng tiết kiệm được thời gian, đồng thời tạo nên một phương thức giao tiếp hỗ trợ chăm sóc sức khỏe thông minh trong đời sống hiện đại.

Chương 1. TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu về đề tài

Chatbot tư vấn y tế là một ứng dụng của công nghệ trí tuệ nhân tạo, tiếp nhận câu hỏi của người dùng liên quan đến các triệu chứng, bệnh lý và các vấn đề về sức khỏe phổ biến, từ đó nó sẽ đưa ra câu trả lời dựa trên cơ sở tri thức y khoa chính thống đã được huấn luyện trước đó. Giải pháp này tuy không thể thay thế được bác sĩ nhưng nó vẫn đóng vai trò là một công cụ hỗ trợ ban đầu, giúp người dùng hiểu rõ tình trạng sức khỏe hiện tại của bản thân hơn. Việc ứng dụng trí tuệ nhân tạo này vào lĩnh vực y tế sẽ góp phần nâng cao chất lượng sức khỏe của cộng đồng.

1.2. Lý do chọn đề tài này

Xem xét tình hình cho thấy, tại Việt Nam vẫn còn rất nhiều người gặp khó khăn trong việc tiếp cận thông tin y tế đáng tin cậy, đặc biệt là ở các vùng nông thôn, vùng sâu, vùng xa hoặc những người không có thời gian để đến cơ sở y tế. Các tài liệu thông tin y tế trên mạng thì đôi khi không phải là nguồn tin chính thống, chưa được xác thực bởi các chuyên gia y khoa gây ra hiểu lầm làm cho người bệnh tự chẩn đoán sai làm nguy hiểm đến tình trạng sức khỏe. Ngoài ra, việc thiếu hụt về nguồn lực y tế như bác sĩ hay nhân viên y tế dẫn đến tình trạng quá tải ở các bệnh viện và trung tâm y tế. Đây là các lý do chính để em thực hiện đề tài này. Phạm vi của đề tài chỉ đơn giản là tập trung vào việc tư vấn, tra cứu kiến thức y tế cơ bản, cung cấp giải pháp phòng ngừa tức thời giúp người dùng đánh giá sơ bộ tình trạng sức khỏe.

1.3. Mục tiêu của đề tài

Mục tiêu mà đề tài hướng đến là xây dựng mô hình chatbot tư vấn y tế cung cấp thông tin y tế đáng tin cậy, dễ tiếp cận, nhanh chóng và thuận tiện nhằm hỗ trợ người dùng muốn tìm hiểu về bệnh lý hay cần một lời khuyên tốt nhất. Từ đó, người dùng có thể tự quản lý sức khỏe, giảm bớt áp lực quá tải cho hệ thống y tế. Và để đạt được những mục tiêu đó thì cần phải xây dựng một chatbot hoạt động ổn định, trả lời chính xác hầu hết các câu hỏi về sức khỏe thông thường, giao diện thân thiện, dễ sử dụng và gần gũi với người dùng. Ngoài việc mang lại trải nghiệm tốt cho người dùng ra, hệ thống còn cung cấp quyền quản trị cho quản trị viên có thể dễ dàng thêm hoặc cập nhật các tài liệu y tế để mở rộng và nâng cao chất lượng cơ sở tri thức của chatbot. Nhờ vậy thì

chatbot mới có thể liên tục được huấn luyện, đảm bảo nguồn dữ liệu mới nhất và độ uy tín cao của thông tin, hướng đến một công cụ tư vấn y tế hiện đại, hữu ích cho cộng đồng.

1.4. Phương pháp thực hiện

Khảo sát thực trạng nhằm thu thập dữ liệu về tình hình y tế hiện nay, tại Việt Nam hay nhiều quốc gia trên thế giới, nhu cầu tìm kiếm thông tin sức khỏe của người dùng chưa bao giờ là giảm sút. Một số nền tảng chăm sóc sức khỏe trực tuyến và ứng dụng tư vấn y tế đều đã được triển khai, nhưng đa phần chỉ dừng lại ở hình thức tra cứu thông tin hay đặt lịch khám bệnh. Việc áp dụng trí tuệ nhân tạo thông minh vào tư vấn sức khỏe còn chưa phổ biến vì còn nhiều lý do như sự hạn chế về ngôn ngữ tự nhiên, độ chính xác khi xử lý dữ liệu y tế hay khả năng cá nhân hóa trải nghiệm người dùng. Nhiều chatbot hiện nay vẫn còn hoạt động theo quy tắc có sẵn (rule-based), thiếu tính linh hoạt và chưa đáp ứng được nhu cầu trao đổi tự nhiên, đa dạng của người dùng.

Để khắc phục những hạn chế trên, mô hình chatbot tư vấn y tế được đề xuất triển khai dựa trên sự kết hợp của trí tuệ nhân tạo và xử lý ngôn ngữ tự nhiên (NLP), cụ thể:

- Ứng dụng mô hình Retrieval-Augmented Generation (RAG): RAG cho phép chatbot không chỉ dựa vào kiến thức có sẵn của mô hình ngôn ngữ mà còn truy xuất trực tiếp từ các tài liệu, báo cáo, cơ sở dữ liệu y khoa,... cách tiếp cận này giúp nâng cao độ chính xác của câu trả lời và giảm thiểu rủi ro khi thông tin bị sai lệch.
- Sử dụng LangChain và ChromaDB để quản lý kho dữ liệu tri thức:
 - + LangChain hỗ trợ xử lý với nhiều tài liệu đa dạng (PDF, CSV, văn bản...) và chia nhỏ dữ liệu đó thành các đoạn có ngữ cảnh rõ ràng.
 - + Các vector embedding được lưu trữ trong vector store (ChromaDB) dùng để tìm kiếm nhanh chóng, giúp chatbot tìm ra thông tin liên quan trong kho dữ liệu lớn.
- Mô hình ngôn ngữ lớn (LLM): Sử dụng mô hình LLM thông qua API của Fireworks AI làm nền tảng xử lý ngôn ngữ tự nhiên và sinh câu trả lời. Mô hình này có khả năng hiểu ngữ cảnh và đưa ra phản hồi tự nhiên, thân thiện phù hợp cho giao tiếp trong y tế.
- Xây dựng kiến trúc hệ thống client - server:

- + Backend: triển khai bằng Django REST Framework, xử lý logic, quản lý phiên trò chuyện và phân quyền người dùng.
- + Frontend: phát triển bằng ReactJS, cung cấp giao diện thân thiện, trực quan để người dùng dễ dàng trao đổi với chatbot.
- + Quản trị tri thức: quản trị viên có thể thêm hoặc cập nhật tài liệu y khoa để hệ thống tự động xử lý và bổ sung vào vector store.
- An toàn và bảo mật: Toàn bộ thông tin người dùng được lưu trữ và bảo mật cẩn thận trong cơ sở dữ liệu MySQL. Chatbot chỉ đóng vai trò cung cấp thông tin tham khảo, không thể thay thế bởi chẩn đoán y khoa đồng thời tuân thủ quy chuẩn đạo đức và pháp lý trong lĩnh vực y tế.

1.5. Bố cục đề tài

Chương 1 – Tổng quan đề tài: Đặt nền tảng lý thuyết và định hướng rõ ràng để phát triển mô hình chatbot trong lĩnh vực y tế, đưa ra lý do, mục tiêu và phương pháp thực hiện cho đề tài.

Chương 2 – Cơ sở lý thuyết: Làm rõ cơ sở lý thuyết, lý do sử dụng và cách triển khai các công nghệ trong đề tài.

Chương 3 – Hệ thống Chatbot tư vấn y tế: Thực hiện phân tích và thiết kế mô hình hệ thống.

Chương 4 – Kết luận và hướng phát triển: Đưa ra những kết luận tổng quan về đề tài và những hướng phát triển trong tương lai.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Chatbot AI

Chatbot được chia làm hai loại chính gồm chatbot quy tắc (rule-based chatbot) và chatbot AI:

- Chatbot quy tắc là loại chatbot đơn giản nhất, chỉ dựa trên những quy tắc hay kịch bản đã được lập trình sẵn để trả lời người dùng theo các từ khóa tương ứng.
- Chatbot AI là một ứng dụng của trí tuệ nhân tạo (AI) được thiết kế để mô phỏng cuộc trò chuyện và tương tác trực tuyến với con người thông qua giọng nói hoặc văn bản. Chatbot sẽ sử dụng thuật toán học máy (Machine Learning) kết hợp ngôn ngữ tự nhiên (NLP) để đưa ra các phản hồi chính xác cho người dùng dựa vào nội dung và ngữ cảnh cuộc trò chuyện hiện tại của họ. Nhờ khả năng có thể tự động trả lời và xử lý yêu cầu, công cụ này mang đến nhiều lợi ích trong nhiều lĩnh vực khác nhau để phục vụ đời sống của con người.

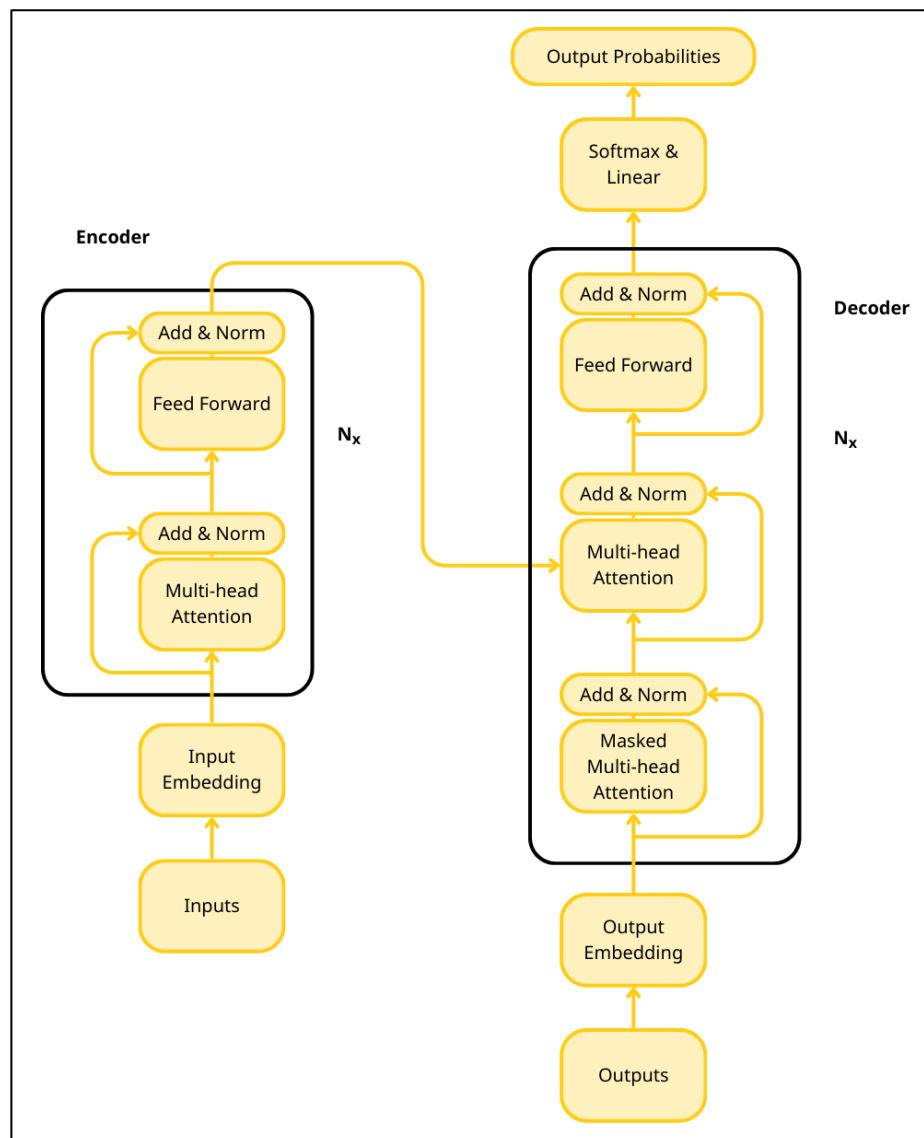
Phát triển chatbot AI giúp doanh nghiệp cũng như người dùng có thêm nhiều lợi ích thiết thực. Sự xuất hiện của chatbot sẽ làm giảm chi phí nhân sự bằng cách tự động hóa các tác vụ lặp đi lặp lại, từ đó tối ưu hiệu quả công việc và nâng cao quy trình kinh doanh. Chatbot làm việc 24/7, phản hồi nhanh chóng và chính xác giúp khách hàng nhận được hỗ trợ kịp thời mà không cần chờ đợi nhân viên nhờ vậy mà trải nghiệm của khách hàng được nâng cao rõ rệt. Nhờ dữ liệu được lập trình sẵn, chatbot đảm bảo độ chính xác và sự nhất quán trong câu trả lời, đồng thời mang đến sự linh hoạt khi có thể tích hợp được vào nhiều nền tảng khác nhau như mạng xã hội, website hay ứng dụng nhắn tin cho phép doanh nghiệp tiếp cận đến nhiều khách hàng và phục vụ nhu cầu đa dạng của họ.

Để xây dựng chatbot AI, đầu tiên chúng ta cần phải xác định mục tiêu, phạm vi ứng dụng và đối tượng sử dụng, sau đó chuẩn bị nguồn dữ liệu đáng tin cậy. Các tài liệu được thu thập, xử lý và chuyển đổi thành vector embeddings rồi lưu trữ trong cơ sở dữ liệu vector để hỗ trợ tìm kiếm ngữ nghĩa. Chatbot ứng dụng mô hình ngôn ngữ lớn (LLM) kết hợp với cơ chế RAG (Retrieval-Augmented Generation) nhằm đảm bảo câu trả lời vừa tự nhiên vừa chính xác, đồng thời sử dụng LangChain để quản lý quy trình hỏi đáp, tạo prompt và duy trì bối cảnh hội thoại.^[1]

2.1.1. Mô hình ngôn ngữ lớn (Large Language Model – LLM)

2.1.1.1. Giới thiệu về LLM

Mô hình ngôn ngữ lớn (LLM) là một dạng trí tuệ nhân tạo được huấn luyện để hiểu và tạo ra ngôn ngữ tự nhiên (NLP) gần giống với con người dựa trên khối lượng dữ liệu văn bản rất lớn từ hàng tỷ đến hàng trăm tỷ tham số. Các LLM chủ yếu sử dụng kiến trúc Transformer và được xây dựng dựa trên công nghệ học sâu (Deep Learning) bao gồm hai thành phần chính: bộ mã hóa và bộ giải mã, hoạt động với cơ chế tự chú ý (self-attention). Hai thành phần này giúp mô hình nắm bắt ý nghĩa, ngữ cảnh của các chuỗi văn bản và mối quan hệ giữa các từ trong văn bản, bất kể độ dài khác nhau. Nhờ đó, LLM không chỉ xử lý các câu hỏi đơn giản mà còn thực hiện nhiều tác vụ ngôn ngữ khác nhau như dịch thuật, tóm tắt, sáng tạo nội dung, hỗ trợ lập trình,...^[2]



Hình 2.1. Kiến trúc Transformer của mô hình LLM

Như đã giới thiệu ở trên, LLM được xây dựng dựa trên kiến trúc Transformer. Kiến trúc này giúp mô hình có thể hiểu và xử lý thông tin dữ liệu hiệu quả, đặc biệt phù hợp với các tác vụ xử lý ngôn ngữ tự nhiên. Transformer có hai thành phần chính gồm bộ mã hóa (encoder) và bộ giải mã (decoder). Bộ mã hóa sẽ đảm nhận xử lý chuỗi văn bản đầu vào và mã hóa chúng thành các vector ngữ nghĩa. Sau đó, bộ giải mã sử dụng thông tin từ encoder và các token đã sinh ra trước đó để tạo ra chuỗi văn bản đầu ra.^[3]

- Bộ mã hóa (encoder): Đầu vào sẽ là chuỗi các token, mỗi token được ánh xạ thành các vector thông qua lớp embedding, kèm theo thông tin vị trí (positional encoding) để giữ thứ tự. Sau đó, chuỗi vector này đi qua N_x lớp lặp lại (thường sẽ là 6 đến 12 lớp trong transformer gốc, hàng chục đến hàng trăm lớp LLM), mỗi lớp gồm có 3 bước: đầu tiên là Multi-head Attention giúp mỗi từ có thể “chú ý”, cho phép mỗi từ tập trung vào các từ khác trong câu để nắm bắt được ngữ cảnh. Các thành phần chính của cơ chế chú ý gồm: Query (Truy vấn) là các phần tử trong chuỗi văn bản đầu vào mà mô hình đang xem xét mức độ liên quan, Key (Khóa) là phần tử trong chuỗi văn bản đầu vào được dùng để đối chiếu sự liên quan, Value (Giá trị) là thông tin đầu ra được tính bằng cách kết hợp các giá trị theo trọng số mà cơ chế chú ý xác định; tiếp theo là Add & Norm (residual connection và layer normalization) thực hiện cộng đầu ra của attention với input ban đầu, sau đó chuẩn hóa để ổn định huấn luyện; cuối cùng là Feed Forward, đây là một mạng nơ-ron fully connected áp dụng cho từng token cho phép biến đổi phi tuyến để tăng khả năng biểu diễn.
- Bộ giải mã (decoder): Quá trình bắt đầu bằng cách biến đổi các token đầu ra trước đó thành vector embedding, các vector này trước hết phải đi qua một lớp Masked Multi-head Attention đảm bảo rằng mỗi từ chỉ được nhìn thấy những từ trước đó chứ không “nhìn trước tương lai”. Sau đó, decoder sử dụng một lớp Multi-head Attention khác để kết hợp thông tin từ encoder giúp liên kết giữa ngôn ngữ nguồn và ngôn ngữ đích. Feed Forward cùng với các bước chuẩn hóa tương tự như encoder. Các lớp này cũng được lặp lại N_x lần để mô hình học sâu hơn.
- Sau cùng thì đầu ra của decoder đi qua Softmax và một lớp tuyến tính (Linear) để cho ra phân phối xác suất trên toàn bộ từ vựng. Mô hình sẽ chọn ra token có

xác suất cao nhất để sinh tiếp từ mới. Quá trình này lặp lại cho đến khi tạo thành toàn bộ chuỗi đầu ra.

2.1.1.2. Các thành phần cơ bản của LLM và lý do sử dụng

Một mô hình ngôn ngữ lớn có cấu tạo từ nhiều lớp mạng nơ-ron (neural network). Trong đó mỗi lớp có một chức năng riêng biệt và phối hợp với nhau để xử lý văn bản đầu vào và tạo ra kết quả đầu ra mong muốn. Gồm 4 lớp cơ bản: embedding, feedforward, recurrent, attention và 3 loại mô hình chính: mô hình ngôn ngữ cơ bản (generic/raw), mô hình ngôn ngữ có hướng dẫn (instruction-tuned), mô hình ngôn ngữ dành cho đối thoại (dialog-tuned).^[4]

Sử dụng mô hình ngôn ngữ lớn trong lĩnh vực y tế giúp tăng tính linh hoạt, không bị giới hạn trong các kịch bản cố định như các hệ thống chatbot truyền thống chỉ dựa trên quy tắc (rule-based) khi đối mặt với những câu hỏi mang tính đa dạng, phức tạp và cần câu trả lời tự nhiên, dễ hiểu cho người dùng.

2.1.1.3. Triển khai mô hình

Trong đồ án này, em sử dụng mô hình GPT-OSS-120B thông qua dịch vụ Fireworks AI. Đây là mô hình ngôn ngữ lớn với hơn 120 tỷ tham số có khả năng tạo ra ngôn ngữ tự nhiên chất lượng cao. Mô hình được tích hợp trực tiếp trong hệ thống thông qua API ChatFireworks, đóng vai trò là thành phần cốt lõi để xử lý và phản hồi câu trả lời cho người dùng.

2.1.2. Retrieval-Augmented Generation

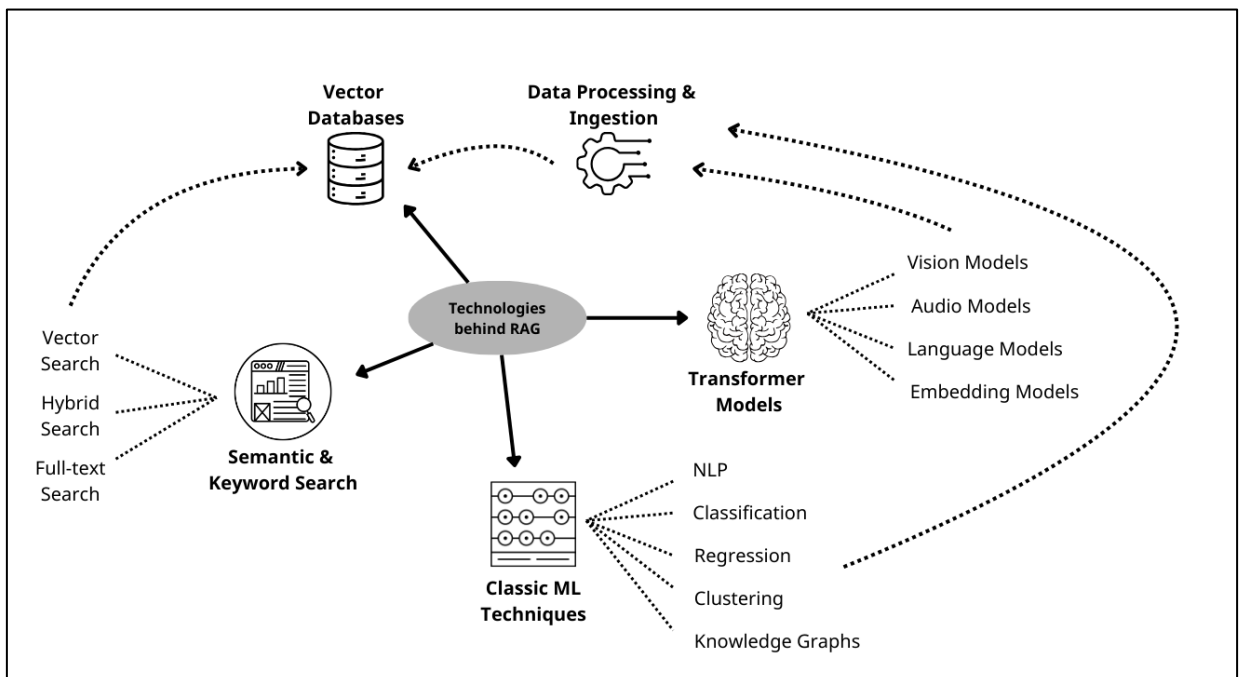
2.1.2.1. Giới thiệu về Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) là một kỹ thuật kết hợp giữa hai thành phần chính: mô hình tìm kiếm thông tin (retriever) và mô hình sinh ngôn ngữ (generator). Thay vì để mô hình ngôn ngữ trả lời trực tiếp dựa trên kiến thức đã huấn luyện, RAG cho phép tham khảo một cơ sở kiến thức có thẩm quyền bên ngoài các nguồn dữ liệu đào tạo của nó trước khi tạo phản hồi. Với cách tiếp cận này sẽ giúp LLM không chỉ dựa vào kiến thức cố định trong quá trình huấn luyện mà còn có thể cập nhật và mở rộng kiến thức theo thời gian thông qua cơ sở dữ liệu được cung cấp.^[5]

2.1.2.2. Lý do sử dụng

Như chúng ta đã biết thì LLM là một công nghệ trí tuệ nhân tạo quan trọng, hỗ trợ các chatbot thông minh, các ứng dụng trong việc sinh ra và xử lý ngôn ngữ tự nhiên, mục đích chính tạo ra chatbot là có thể trả lời các câu hỏi của người dùng bằng cách tham chiếu chéo đến các nguồn kiến thức chính thống. Nhưng thực chất công nghệ LLM tạo ra sự không thể đoán trước trong phản hồi của LLM. Ngoài ra, dữ liệu đào tạo LLM là tĩnh, dẫn đến những hạn chế về thời gian đối với kiến thức hiện có, đặt ra nhiều thách thức lớn cho LLM như trình bày thông tin sai lệch khi không có câu trả lời, thông tin lỗi thời hoặc trả lời chung chung khi người dùng mong đợi một phản hồi cụ thể, tạo phản hồi không chính xác do nhầm lẫn thuật ngữ hay khi các nguồn đào tạo khác nhau sử dụng cùng một thuật ngữ để chỉ những điều khác nhau.

Vì vậy, RAG là một cách để giải quyết những thách thức này. Nó sẽ chuyển hướng các LLM để truy xuất thông tin liên quan từ các nguồn kiến thức được xác định trước và có thẩm quyền. Các tổ chức có khả năng kiểm soát tốt hơn đối với đầu ra văn bản được tạo ra và người dùng sẽ có cái nhìn chi tiết hơn về cách mà LLM tạo ra phản hồi.



Hình 2.2. Mô hình RAG và các công nghệ phía sau nó

Bên trên là hình ảnh mô tả mô hình RAG và các công nghệ phía sau nó^[6]:

- Các mô hình Transformer (ngôn ngữ, hình ảnh, âm thanh, embedding) sinh ra dữ liệu đặc trưng hoặc embeddings. Những dữ liệu này được đưa về bước Data

Processing & Ingestion để tiếp tục xử lý, chuẩn hóa hoặc lưu trữ. (*Transformer Models → Data Processing & Ingestion*)

- Các embedding vectors sẽ được lưu vào cơ sở dữ liệu vector, cho phép tìm kiếm theo ngữ nghĩa tức là thông tin liên quan chứ không chỉ là khớp từ khóa. (*Data Processing & Ingestion → Vector Databases*)
- Khi có một câu hỏi (query) từ người dùng thì câu hỏi sẽ được chuyển thành embedding, sau đó thực hiện tìm kiếm trên Vector Database để lấy ra các vector hoặc dữ liệu liên quan. (*Semantic & Keyword Search → Vector Databases*)
- Các kỹ thuật ML truyền thống (NLP, classification, regression, clustering, knowledge graphs) tạo ra nhãn, đặc trưng hoặc cấu trúc dữ liệu. Những kết quả này quay trở lại Data Processing & Ingestion để làm giàu thêm dữ liệu trước khi đưa vào Vector Databases. (*Classic ML Techniques → Data Processing & Ingestion*)

2.1.2.3. Triển khai trong đề tài

Cơ chế RAG được triển khai bằng cách kết hợp giữa retriever và generator:

- Retriever: Sử dụng ChromaDB làm vector store để tìm kiếm các đoạn tài liệu liên quan đến câu hỏi của người dùng.
- Generator: Mô hình GPT-OSS-120B (Fireworks AI) sẽ đảm nhận đầu vào là các đoạn thông tin đã truy xuất sau đó tổng hợp và sinh ra câu trả lời liên quan.

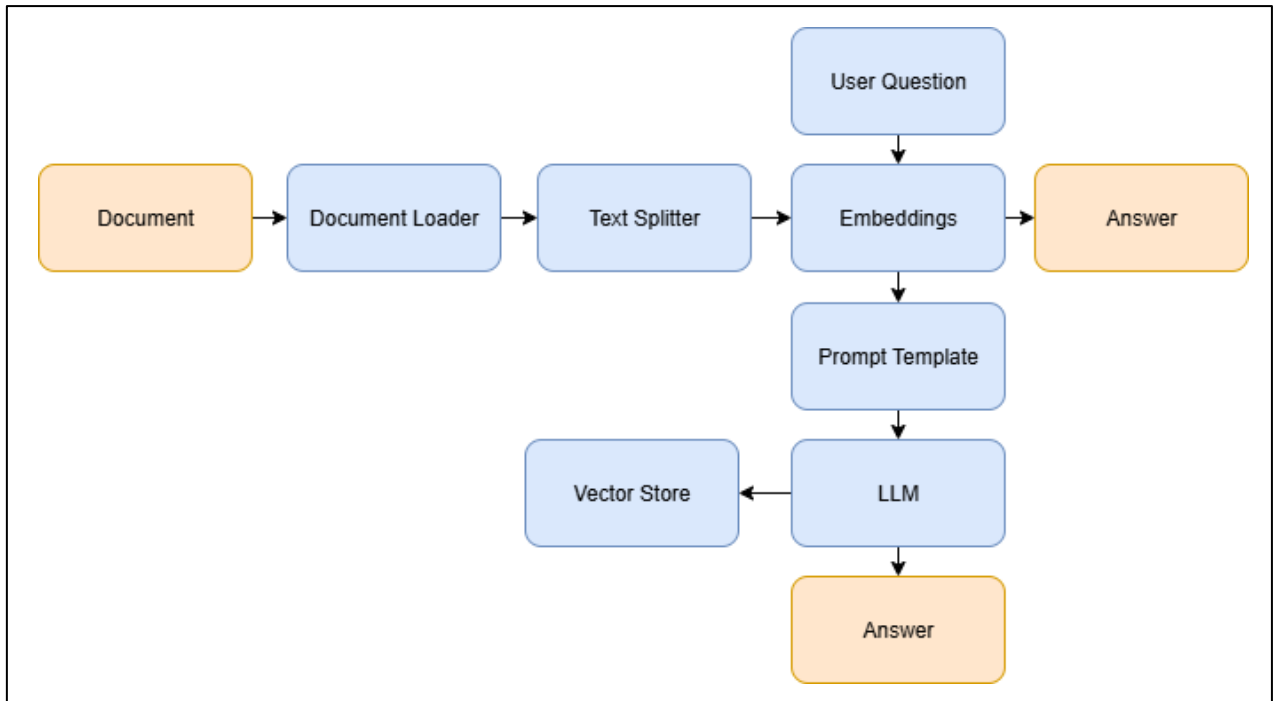
Quá trình này được thực hiện nhờ Conversational Retrieval Chain trong thư viện LangChain, kết hợp với Chat Prompt Template để điều chỉnh cách đặt câu hỏi và cấu trúc phản hồi.

2.1.3. LangChain

2.1.3.1. Giới thiệu về LangChain

LangChain là một framework mã nguồn mở cung cấp bộ công cụ và thành phần để xây dựng nên các ứng dụng dựa trên mô hình ngôn ngữ lớn. Nó giúp các nhà phát triển đơn giản hóa mọi giai đoạn trong vòng đời của ứng dụng LLM như giảm độ phức tạp khi kết nối nhiều thành phần LLM, embedding, vector database và RAG, đồng thời cung cấp sẵn các module loader, splitter, retriever, chain để làm tăng tốc độ phát triển.

2.1.3.2. Kiến trúc LangChain



Hình 2.3. Kiến trúc của LangChain

Document Processing (Xử lý dữ liệu đầu vào):

- Document: Tài liệu thô (PDF, Word, HTML, API data,...).
- Document Loader: Bộ nạp tài liệu dùng để đọc dữ liệu từ nhiều nguồn khác nhau (file, database, web,...).
- Text Splitter: Chia nhỏ tài liệu thành các đoạn (chunks) để dễ quản lý, tìm kiếm và embedding.

Embeddings (Biểu diễn vector): Các đoạn văn bản sau khi chia nhỏ sẽ được biến đổi thành vector embeddings bằng mô hình embedding, các vector này mang thông tin ngữ nghĩa và sẽ được lưu vào Vector Store để phục vụ cho việc tìm kiếm ngữ nghĩa.

User Question (Câu hỏi từ người dùng): Người dùng đặt câu hỏi, câu hỏi được chuyển thành embedding vector để tìm kiếm nội dung liên quan trong Vector Store. Sau đó, câu hỏi và dữ liệu truy xuất sẽ được đưa vào Prompt Template.

Prompt Template (Xây dựng ngữ cảnh): Đây là bước định nghĩa “cách đặt câu hỏi” cho LLM.

LLM (Ngôn ngữ lớn): Mô hình ngôn ngữ nhận prompt đã được chuẩn bị. Nó dùng dữ liệu bổ sung từ Vector Store để sinh ra câu trả lời có ngữ cảnh và chính xác hơn.

Answer: Kết quả cuối cùng được sinh ra từ LLM trả về cho người dùng dưới dạng phản hồi.

LangChain hoạt động theo một luồng tổng thể như sau: Đầu tiên, các tài liệu gốc như văn bản, PDF, cơ sở dữ liệu hoặc nội dung web sẽ được nạp vào hệ thống thông qua Document Loader. Sau đó, chúng được Text Splitter chia nhỏ thành các đoạn văn bản ngắn hơn để dễ xử lý, các đoạn này được biến đổi thành vector embeddings bằng mô hình embedding và lưu trữ trong Vector Store nhằm phục vụ cho việc tìm kiếm ngữ nghĩa. Khi người dùng đặt một câu hỏi, truy vấn đó cũng được chuyển thành embedding và đối chiếu trong Vector Store để tìm ra những đoạn nội dung liên quan nhất. Tiếp theo, LangChain sử dụng Prompt Template để kết hợp câu hỏi người dùng với dữ liệu tìm được hình thành một ngữ cảnh đầy đủ, ngữ cảnh này được gửi đến LLM để tạo ra câu trả lời. Cuối cùng, hệ thống trả về Answer cho người dùng, đảm bảo kết quả không chỉ dựa trên kiến thức của mô hình mà còn gắn với dữ liệu thực tế đã nạp vào.^[7]

2.1.3.3. Triển khai LangChain

- Xây dựng pipeline hỏi đáp: Kết hợp retriever từ ChromaDB và LLM thông qua Conversational Retrieval Chain. Chain này sẽ tự động lấy tài liệu phù hợp, ghép với câu hỏi và lịch sử hội thoại sau đó chuyển cho LLM xử lý.
- Quản lý prompt: Dùng Chat Prompt Template để định nghĩa mẫu nhắc nhở (prompt).
- Quản lý hội thoại: Lưu lịch sử câu hỏi và câu trả lời tương ứng để truyền lại cho LLM qua chain. Nhờ đó, chatbot có thể duy trì được bối cảnh của cuộc hội thoại và phản hồi một cách tự nhiên hơn.
- Nạp dữ liệu vào kho: Sử dụng Directory Loader để tự động duyệt thư mục dữ liệu. Sau khi nạp, văn bản sẽ được xử lý bởi Recursive Character Text Splitter để chia thành các đoạn theo ký tự, đảm bảo mỗi đoạn có độ dài dưới một độ dài nhất định, hữu ích cho các tài liệu có ngắt đoạn hoặc ngắt câu tự nhiên.

2.2. Django

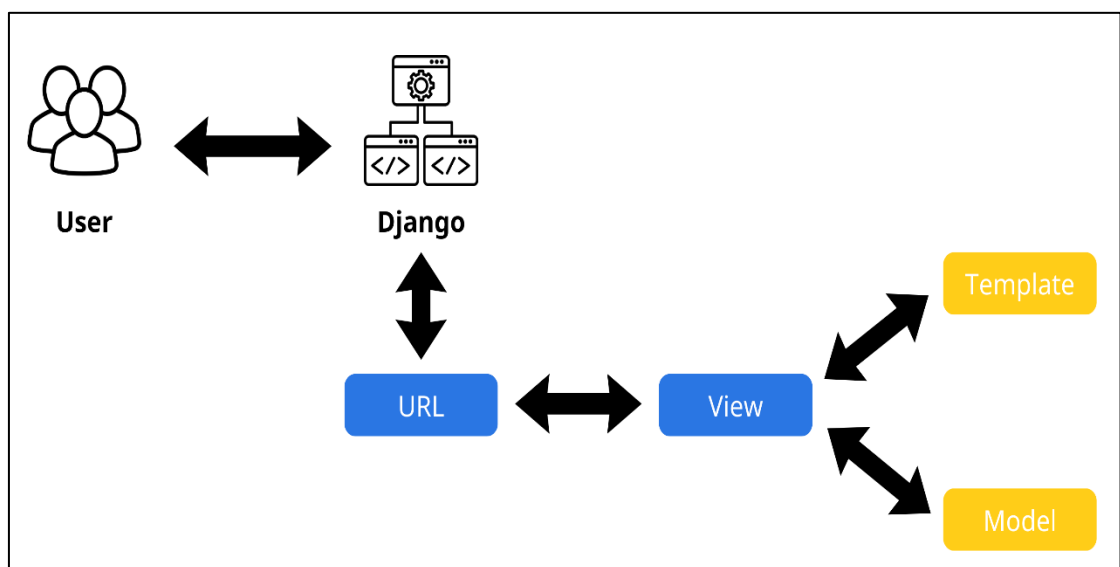
2.2.1. Giới thiệu Django

Django là một framework được sử dụng rộng rãi để phát triển ứng dụng web một cách nhanh chóng và hiệu quả. Nó cung cấp sẵn nhiều chức năng phổ biến như xác thực, quản lý cơ sở dữ liệu,... dưới dạng các mô-đun tái sử dụng. Nhờ đó, lập trình viên có thể tổ chức mã nguồn tốt hơn và rút ngắn đáng kể thời gian phát triển ứng dụng web.^[8]

2.2.2. Kiến trúc Django

Kiến trúc của Django dựa trên mô hình MVT (Model – View – Template), khá giống với MVC (Model – View – Controller) nhưng có một số khác biệt:

- Model: Đại diện cho lớp dữ liệu (database), chịu trách nhiệm định nghĩa cấu trúc bảng, các trường dữ liệu và mối quan hệ giữa chúng. Django cung cấp ORM (Object Relational Mapping), cho phép thao tác với cơ sở dữ liệu bằng Python thay vì SQL trực tiếp.
- View: Xử lý logic nghiệp vụ của ứng dụng, nhận yêu cầu từ người dùng, gọi đến Model để lấy dữ liệu cần thiết sau đó thì chuyển dữ liệu cho Template để hiển thị.
- Template: Chịu trách nhiệm hiển thị dữ liệu trên giao diện người dùng (UI) mà View truyền đến. Được viết bằng DTL (Django Template Language) và cho phép nhúng logic đơn giản (if, for, filter) vào HTML.



Hình 2.4. Kiến trúc của Django

Hình trên minh họa cơ chế hoạt động của Django theo mô hình MVT. Người dùng gửi yêu cầu thông qua trình duyệt đến hệ thống Django, thông tin này sẽ được định tuyến qua URL và chuyển tiếp đến View để xử lý logic nghiệp vụ. View sẽ truy xuất dữ liệu từ Model và gửi dữ liệu đến Template để hiển thị ra giao diện HTML. Cuối cùng, kết quả phản hồi được trả về cho người dùng.

Nhờ cơ chế hoạt động này, Django đảm bảo sự tách biệt rõ ràng giữa các thành phần: Model quản lý dữ liệu, View xử lý logic và Template hiển thị giao diện làm cho việc phát triển ứng dụng web trở nên dễ dàng, có tổ chức và dễ bảo trì.^[9]

2.2.3. Triển khai Django

RESTful API là một tiêu chuẩn dùng để thiết kế API cho các ứng dụng web cho phép các hệ thống giao tiếp và trao đổi dữ liệu với nhau thông qua giao thức HTTP dựa trên nguyên tắc của REST. Ngoài ra, Django REST Framework – một công cụ mạnh mẽ và linh hoạt còn hỗ trợ mạnh mẽ cho việc xây dựng API bằng cách cung cấp đầy đủ các phương thức như GET, POST, PUT, PATCH, DELETE để thao tác dữ liệu, đồng thời truyền tải dữ liệu dưới dạng phổ biến như JSON hoặc XML. Đây là công cụ quan trọng để xây dựng các ứng dụng web hiện đại, đảm bảo tính linh hoạt và hiệu quả trong trao đổi dữ liệu giữa các thành phần trong hệ thống.



Hình 2.5. Kiến trúc của RESTful API

Bên trên là hình ảnh kiến trúc của RESTful API: Phía giao diện người dùng sẽ gửi yêu cầu (request) bằng các phương thức GET, POST, PUT, DELETE,... về phía server, sau đó server sẽ xử lý và trả về kết quả (response) dưới dạng JSON, XML, HTML,... cho phía giao diện người dùng. Mọi sự giao tiếp giữa giao diện người dùng và server này đều thông qua RESTful API.

Django cung cấp kiến trúc MVT rõ ràng, tích hợp ORM, hệ thống authentication và quản lý cơ sở dữ liệu mạnh mẽ, phù hợp cho việc xây dựng hệ thống có nhiều người dùng và phân quyền. DRF hỗ trợ triển khai RESTful API nhanh chóng với serializer,

viewset, router, đồng thời tích hợp sẵn các cơ chế authentication, permission, pagination. RESTful API là chuẩn thiết kế API phổ biến, dễ dàng phát triển song song và tích hợp với các hệ thống khác. Việc kết hợp Django, DRF và RESTful API sẽ mang lại sự cân bằng giữa tốc độ phát triển, bảo mật và khả năng mở rộng.^[10]

Trong đề án này, Django được kết hợp cùng Django REST Framework để xây dựng RESTful API phục vụ cho hệ thống chatbot tư vấn y tế. Django đóng vai trò là backend xử lý logic nghiệp vụ, quản lý dữ liệu và phân quyền người dùng, trong khi DRF hỗ trợ xây dựng các API trả về dữ liệu dưới dạng JSON để frontend có thể giao tiếp dễ dàng. Cách tiếp cận này cho phép tách biệt frontend và backend, giúp hệ thống linh hoạt, dễ mở rộng và tích hợp thêm dịch vụ khác trong tương lai.

2.3. ReactJS

2.3.1. Giới thiệu ReactJS

ReactJS hay còn được gọi là React, đây là một thư viện JavaScript mã nguồn mở mạnh mẽ được phát triển bởi Facebook hỗ trợ phát triển giao diện người dùng cho các ứng dụng web hiện đại với khả năng tương tác cao. Nó cho phép người dùng nhúng code HTML trong code Javascript nhờ vào JSX. ReactJS chỉ tập trung vào phần hiển thị giao diện người dùng chứ không can thiệp vào logic nghiệp vụ.^[11]

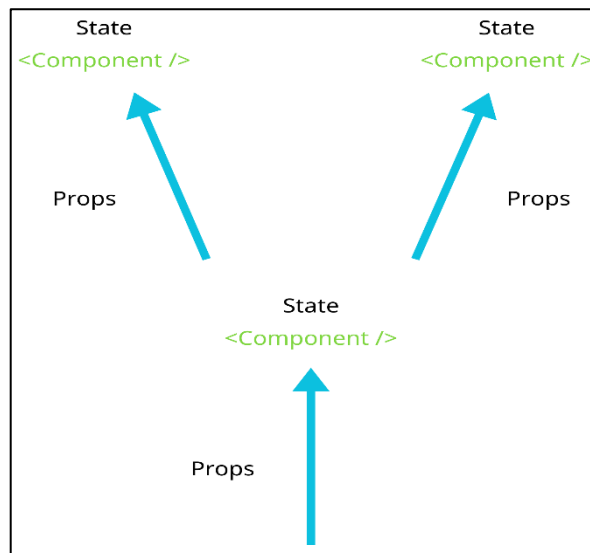
2.3.2. Kiến trúc ReactJS

ReactJS được xây dựng dựa trên kiến trúc component, đây là điểm đặc trưng và cũng là sức mạnh lớn nhất của nó.

Component là một khối giao diện độc lập, có thể tái sử dụng nhiều lần. Mỗi component quản lý riêng logic và giao diện của nó, giúp chia ứng dụng thành các phần nhỏ dễ kiểm soát. Có 2 loại chính: Functional Component và Class Component.

Các thành phần trong kiến trúc component^[12]:

- Props (properties): Có cơ chế truyền từ cha sang con, giúp component trở nên linh hoạt và có thể tái sử dụng với nhiều cấu hình khác nhau.
- State: Kho lưu trữ dữ liệu của các component, có thể thay đổi theo thời gian. Khi state thay đổi, component sẽ render lại để cập nhật giao diện.



Hình 2.6. Props và State trong việc quản lý dữ liệu

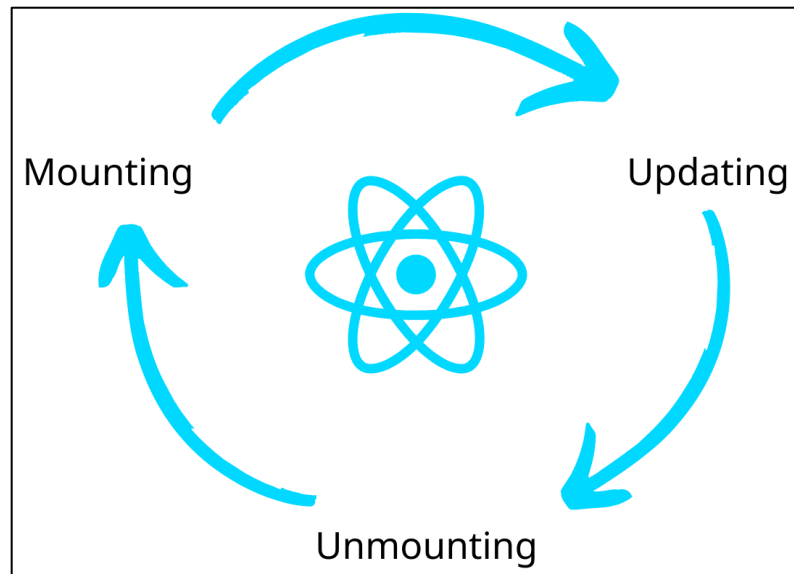
Props (properties) đóng vai trò như các tham số được truyền từ component cha xuống component con, mang tính bất biến (immutable) tức là không thể bị thay đổi bởi component nhận. Cơ chế này cho phép tái sử dụng component với nhiều cấu hình khác nhau, tăng tính linh hoạt của hệ thống. Ngược lại, state là dữ liệu nội bộ được quản lý trong từng component. Khi giá trị của state thay đổi, ReactJS sẽ tự động kích hoạt quá trình re-render để cập nhật giao diện, đảm bảo dữ liệu và hiển thị luôn đồng bộ. State mang tính cục bộ, chỉ thuộc về component khai báo và không thể trực tiếp truy cập từ bên ngoài. Như vậy, một component trong ReactJS có thể đồng thời nhận dữ liệu từ bên ngoài thông qua props và quản lý dữ liệu bên trong thông qua state. Sự kết hợp này giúp ReactJS duy trì tính nhất quán của dữ liệu, đồng thời cho phép phát triển giao diện động, trực quan và dễ bảo trì.

- Lifecycle (Vòng đời component): Là quá trình từ khi khởi tạo (mounting), thay đổi (updating) và hủy bỏ (unmounting) component. Với functional component, ReactJS cung cấp Hook như useEffect, useState để quản lý vòng đời.

Mounting là giai đoạn khởi tạo, khi component được tạo mới và gắn vào cây DOM. Trong giai đoạn này, ReactJS sẽ gọi các phương thức khởi tạo và các hook như useEffect để thiết lập dữ liệu hoặc thực hiện những thao tác ban đầu.

Updating diễn ra khi dữ liệu thay đổi do sự biến đổi của props hoặc state. Khi đó, React sẽ tiến hành re-render component để hiển thị dữ liệu mới. Đây cũng là giai đoạn mà các phương thức hoặc hook theo dõi sự thay đổi được kích hoạt nhằm xử lý logic liên quan.

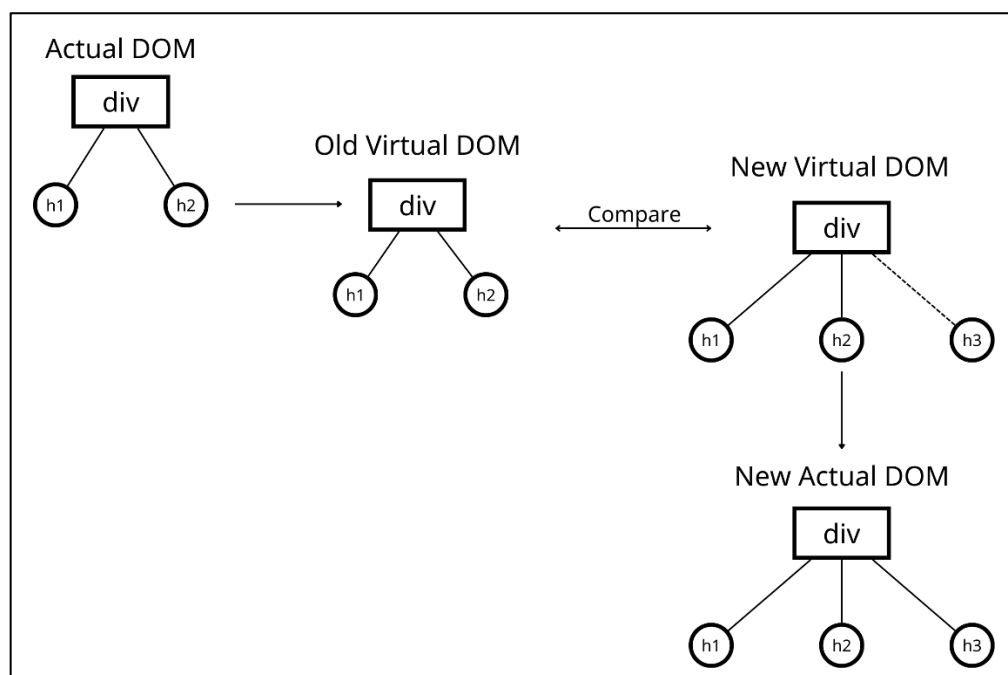
Unmounting là giai đoạn cuối cùng khi component bị loại bỏ khỏi cây DOM. Trong bước này, ReactJS cho phép thực hiện các thao tác dọn dẹp như hủy kết nối, xóa timer hoặc giải phóng bộ nhớ thông qua các cơ chế cleanup.



Hình 2.7. Minh họa Lifecycle của một component trong ReactJS

Nhờ vòng đời này, ReactJS có thể kiểm soát chặt chẽ quá trình tồn tại của mỗi component từ khi khởi tạo, cập nhật cho đến khi kết thúc. Điều này đảm bảo hiệu suất của ứng dụng, đồng thời hạn chế rò rỉ tài nguyên và nâng cao tính ổn định trong suốt quá trình vận hành.

- Virtual DOM: Là một bản sao của DOM thật nhưng tồn tại trong bộ nhớ, khi trạng thái (state) hoặc dữ liệu (props) của component bị thay đổi ReactJS sẽ không cập nhật trực tiếp vào DOM thật ngay mà sẽ tạo ra một Virtual DOM mới sau đó so sánh với Virtual DOM cũ bằng thuật toán Diffing và chỉ cập nhật phần khác biệt giữa chúng vào DOM thật.



Hình 2.8. Cách thức hoạt động của ReactJS

2.3.3. Triển khai ReactJS

Trong đồ án này, ReactJS được sử dụng để xây dựng giao diện người dùng (frontend) cho hệ thống chatbot tư vấn y tế. Việc áp dụng ReactJS cho phép tách biệt hoàn toàn phần giao diện với phần xử lý nghiệp vụ, đồng thời tận dụng mô hình phát triển theo hướng component nhằm nâng cao tính tái sử dụng và khả năng bảo trì hệ thống.

Quá trình triển khai ReactJS trong đồ án bao gồm các bước chính: đầu tiên, dự án được khởi tạo bằng công cụ create-react-app, đồng thời cài đặt các thư viện cần thiết như axios để thực hiện gọi API, react-router-dom để quản lý định tuyến và Material UI để xây dựng giao diện. Tiếp theo, cấu trúc thư mục của dự án được tổ chức hợp lý với các phần riêng biệt như components để chứa các thành phần giao diện, pages để quản lý các trang chính, services để định nghĩa các hàm gọi API và context hoặc redux để quản lý state toàn cục. Trong giai đoạn phát triển, các component được thiết kế nhằm hiển thị giao diện chatbot bao gồm ô nhập tin nhắn, vùng hiển thị hội thoại và các thành phần tương tác khác. Các component này sử dụng props để nhận dữ liệu từ bên ngoài và state để quản lý dữ liệu nội bộ, đồng thời kết hợp với các hook như useState và useEffect để xử lý vòng đời và đồng bộ dữ liệu với backend.

Hệ thống frontend sau đó được kết nối với API của backend Django REST Framework thông qua các request HTTP, đảm bảo dữ liệu từ chatbot được phản hồi và hiển thị kịp

thời trên giao diện người dùng. Trong quá trình này, các vấn đề về tải dữ liệu, xử lý lỗi và phản hồi bất đồng bộ đều được quản lý thông qua cơ chế state và hook của ReactJS.

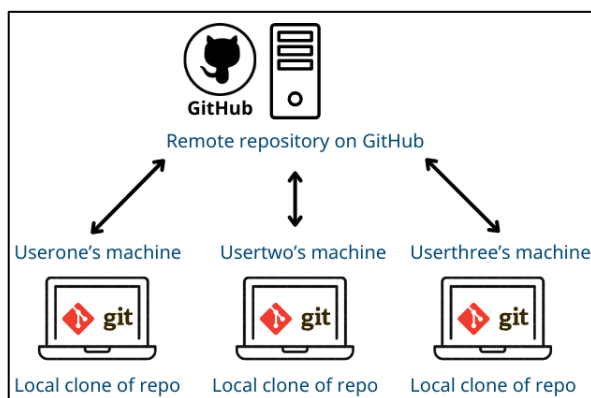
2.4. Các công cụ và thư viện hỗ trợ

2.4.1. MySQL Workbench

MySQL Workbench là công cụ trực quan hỗ trợ làm việc với hệ quản trị cơ sở dữ liệu MySQL. Nó cung cấp một giao diện đồ họa thân thiện, giúp người dùng dễ dàng thiết kế, xây dựng và quản lý cơ sở dữ liệu mà không cần phải thao tác quá nhiều với dòng lệnh để thực thi. Đồng thời tích hợp nhiều tính năng quan trọng như thiết kế mô hình dữ liệu ERD, truy vấn bằng lệnh SQL, theo dõi hiệu năng, quản lý kết nối máy chủ cũng như sao lưu và phục hồi dữ liệu. Sử dụng MySQL Workbench trong đồ án này sẽ giúp em thao tác thuận tiện hơn việc thiết kế, trực quan hóa các bảng và mối quan hệ với nhau từ đó nâng cao tính chính xác và hiệu quả trong quá trình phát triển hệ thống.^[13]

2.4.2. Git/Github

Git và Github đều là những công cụ quan trọng hỗ trợ em trong quá trình phát triển phần mềm. Hai khái niệm này thường đi cùng nhau nhưng thực chất không giống nhau. Git là hệ thống kiểm soát phiên bản giúp lập trình viên theo dõi lịch sử thay đổi của mã nguồn, làm cho hệ thống dễ dàng được quản lý và khôi phục. Github là nền tảng cung cấp dịch vụ lưu trữ và chia sẻ mã nguồn trực tiếp dựa trên Git, hỗ trợ làm việc nhóm hiệu quả thông qua các tính năng như pull request, issue tracking, quản lý nhánh.^[14]



Hình 2.9. Mô tả Github

Sơ đồ này minh họa mô hình hợp tác phát triển phần mềm với Git và GitHub, nơi nhiều người có thể cùng làm việc trên một dự án, quản lý phiên bản hiệu quả và đảm bảo mã nguồn được lưu trữ tập trung, dễ dàng chia sẻ và kiểm soát.

Chương 3. **HỆ THỐNG CHATBOT TƯ VẤN Y TẾ**

3.1. Giới thiệu hệ thống

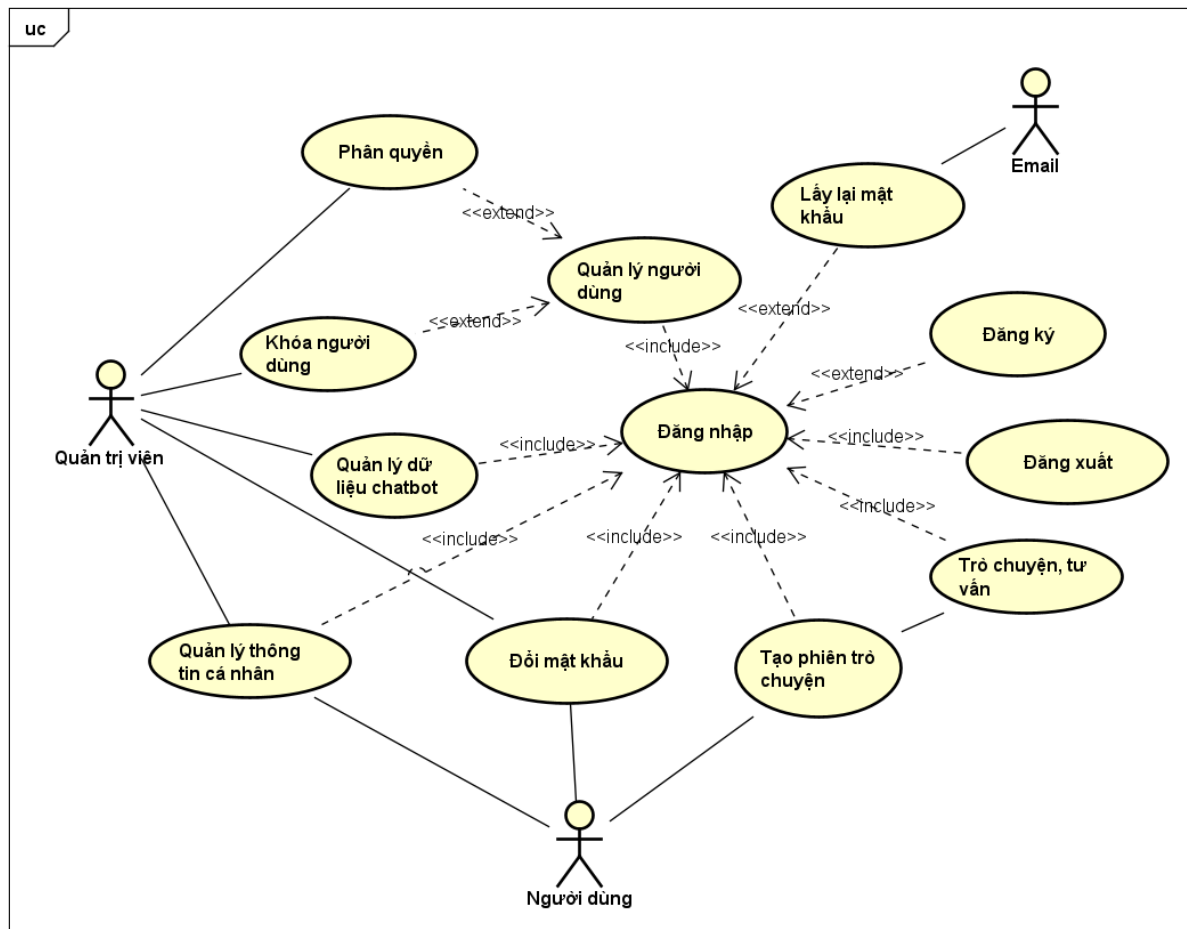
Phát triển hệ thống chatbot tư vấn y tế trong đồ án này nhằm mang đến một công cụ hỗ trợ người dùng tiếp cận thông tin sức khỏe một cách nhanh chóng, chính xác và dễ sử dụng. Chatbot đóng vai trò là cầu nối giữa người dùng và cơ sở tri thức y khoa được tin cậy, giúp người dùng dễ dàng đặt câu hỏi và nhận được phản hồi cụ thể, thân thiện đảm bảo an toàn trong việc tham khảo. Hệ thống cung cấp đầy đủ các chức năng cơ bản cần thiết để phục vụ nhu cầu tìm kiếm thông tin y tế cộng đồng.

Về phía người dùng, giao diện trò chuyện đơn giản và trực quan cho phép đặt câu hỏi liên quan đến triệu chứng, bệnh lý hoặc các vấn đề về sức khỏe. Chatbot sử dụng công nghệ xử lý ngôn ngữ tự nhiên để phân tích nội dung câu hỏi, tìm kiếm thông tin liên quan trong cơ sở dữ liệu y khoa sau đó trả lời ngắn gọn và dễ hiểu. Bên cạnh đó, hệ thống còn ghi nhớ lịch sử hội thoại để làm cho cuộc trò chuyện trở nên liên tục và liền mạch.

Về phía quản trị, hệ thống cho phép thêm mới hoặc cập nhật để quản lý nguồn dữ liệu y khoa được dùng để huấn luyện chatbot. Cơ sở tri thức sẽ luôn được bổ sung, đảm bảo độ chính xác và phù hợp theo từng giai đoạn. Quản trị viên cũng có quyền giám sát hoạt động của chatbot, kiểm tra câu trả lời và cải tiến chất lượng phản hồi.

3.2. Phân tích hệ thống

3.2.1. Lược đồ use case



Hình 3.1. Lược đồ use case

Lược đồ use case trên mô tả các chức năng chính của hệ thống chatbot và mối quan hệ giữa các actor với các use case.

Các tác nhân (Actors):

- Người dùng: sử dụng hệ thống để đăng ký, đăng nhập, trò chuyện và quản lý thông tin cá nhân.
- Quản trị viên: quản lý người dùng, phân quyền, khóa tài khoản và quản lý dữ liệu của chatbot.
- Email: hỗ trợ quá trình lấy lại mật khẩu.

Các use case:

- Đăng ký: Người dùng mới tạo tài khoản để tham gia hệ thống.

- Đăng nhập: Người dùng và quản trị viên phải đăng nhập để sử dụng các chức năng. Đây là use case trung tâm được nhiều chức năng khác bao gồm.
- Đăng xuất: Thoát khỏi hệ thống sau khi sử dụng.
- Trò chuyện, tư vấn: Người dùng thực hiện chức năng chính của chatbot là trò chuyện và nhận tư vấn.
- Tạo phiên trò chuyện: Người dùng khởi tạo một phiên chat mới.
- Đổi mật khẩu: Người dùng thay đổi mật khẩu của tài khoản.
- Lấy lại mật khẩu: Người dùng quên mật khẩu có thể yêu cầu hệ thống gửi email để đặt lại.
- Quản lý thông tin cá nhân: Người dùng cập nhật thông tin cá nhân.
- Quản lý người dùng: Quản trị viên giám sát danh sách người dùng.
- Quản lý dữ liệu chatbot: Quản trị viên thêm, sửa, xóa dữ liệu trong cơ sở tri thức chatbot.

Lược đồ use case có ý nghĩa quan trọng trong việc mô tả các chức năng cốt lõi của hệ thống chatbot dưới góc nhìn của người sử dụng và quản trị viên. Thông qua sơ đồ, ta có thể thấy rõ các tác nhân tham gia hệ thống, những chức năng mà họ có thể thực hiện cũng như mối quan hệ ràng buộc giữa các ca sử dụng. Điều này giúp định hình yêu cầu hệ thống một cách trực quan, dễ hiểu, đồng thời hỗ trợ phát triển và khách hàng có cùng nhận thức về phạm vi và cách thức vận hành của hệ thống. Bên cạnh đó, lược đồ use case cũng đóng vai trò là cơ sở quan trọng để thiết kế chi tiết cơ sở dữ liệu, luồng xử lý và giao diện người dùng trong các giai đoạn tiếp theo của quá trình phát triển.

3.2.2. Đặc tả use case

3.2.2.1. Đăng nhập

Use case ID	UC01_mình
Tên use case	Đăng nhập
Mô tả use case	Chứng thực thông tin username và password của người dùng có thành công hay không
Actor chính	Quản trị viên, Người dùng

Actor phụ	Không có
Tiền điều kiện	- Đã đăng ký tài khoản - Có kết nối mạng Internet
Hậu điều kiện	- Nếu đăng nhập thành công thì vào giao diện trang chủ - Nếu nhập không đúng username hay password sẽ thông báo lỗi và bắt nhập lại
Luồng hoạt động	1. Hiện thị giao diện đăng nhập người dùng 2. Điền các trường bắt buộc: username và password 3. Hệ thống kiểm tra thông tin tài khoản 4. Nếu thông tin hợp lệ thì thông báo đăng nhập thành công 5. Chuyển về trang chủ
Luồng thay thế	4.1. Nếu thông tin không hợp lệ thì thông báo lỗi
Luồng ngoại lệ	Khi hệ thống gặp sự cố thì hiện thị thông báo lỗi

Bảng 3.1. Đặc tả use case chức năng Đăng nhập

3.2.2.2. Đăng ký

Use case ID	UC02_mình
Tên use case	Đăng ký
Mô tả use case	Người dùng đăng ký tài khoản với các thông tin username, password, gmail, ngày sinh, ảnh đại diện, giới tính, họ, tên
Actor chính	Người dùng
Actor phụ	Không có
Tiền điều kiện	Có kết nối mạng Internet
Hậu điều kiện	Người dùng đăng ký thành công chuyển sang đăng nhập
Luồng hoạt động	1. Hiện thị giao diện đăng ký 2. Người dùng nhập các trường bắt buộc như username, password, số điện thoại, gmail, ngày sinh, ảnh đại diện, giới

	tính, họ, tên 3. Nhấn đăng ký 4. Thông tin hợp lệ thì thông báo đăng ký thành công 5. Chuyển sang trang đăng nhập
Luồng thay thế	4.2. Nếu nhập sai định dạng thì thông báo lỗi 4.3. Nếu mật khẩu và xác nhận mật khẩu không trùng khớp thì thông báo lỗi 4.4. Nếu gmail, phone, username đã được đăng ký rồi thì thông báo lỗi thông tin đã tồn tại
Luồng ngoại lệ	Khi hệ thống gặp sự cố thì hiển thị thông báo lỗi

Bảng 3.2. Đặc tả use case chức năng Đăng ký

3.2.2.3. Lấy lại mật khẩu

Use case ID	UC03_mình
Tên use case	Lấy lại mật khẩu
Mô tả use case	Cho phép người dùng đặt lại mật khẩu bằng cách xác thực mã OTP được gửi thông qua gmail mà người dùng nhập vào
Actor chính	Người dùng
Actor phụ	Không có
Tiền điều kiện	- Đã có tài khoản - Có kết nối mạng Internet
Hậu điều kiện	- Người dùng đăng nhập thành công sau khi đặt lại mật khẩu mới
Luồng hoạt động	1. Người dùng chọn quên mật khẩu ở trang đăng nhập 2. Hệ thống yêu cầu người dùng nhập gmail của tài khoản bị quên mật khẩu và nhấn xác nhận 3. Hệ thống sẽ kiểm tra gmail có được đăng ký tài khoản chưa

	4. Nếu gmail có tồn tại thì gửi mã OTP gồm 8 số qua gmail đó 5. Người dùng kiểm tra gmail và nhập đúng mã OTP và nhấn xác nhận 6. Nếu mã OTP đúng thì người dùng nhập mật khẩu mới và xác nhận mật khẩu vừa nhập 7. Thông báo đổi mật khẩu thành công và quay về trang đăng nhập
Luồng thay thế	3.1. Nếu gmail chưa được đăng ký tài khoản thì thông báo lỗi 6.1. Nếu mã OTP sai hoặc hết hạn sử dụng thì thông báo lỗi
Luồng ngoại lệ	Khi hệ thống gặp sự cố thì thông báo lỗi và quay trở về trang đăng nhập

Bảng 3.3. Đặc tả use case chức năng Lấy lại mật khẩu

3.2.2.4. Đăng xuất

Use case ID	UC04_minh
Tên use case	Đăng xuất
Mô tả use case	Thoát khỏi trạng thái đăng nhập
Actor chính	Người dùng, Quản trị viên
Actor phụ	Không có
Tiền điều kiện	- Có kết nối mạng Internet - Đang đăng nhập trong hệ thống
Hậu điều kiện	Đăng xuất thành công
Luồng hoạt động	1. Bấm vào ảnh đại diện người dùng 2. Chọn đăng xuất 3. Đăng xuất thành công 4. Trở về trang đăng nhập
Luồng thay thế	

Luồng ngoại lệ	Khi hệ thống gặp sự cố thì thông báo lỗi
-----------------------	--

Bảng 3.4. Đặc tả chức năng Đăng xuất

3.2.2.5. Quản lý thông tin cá nhân

Use case ID	UC05_mình
Tên use case	Quản lý thông tin cá nhân
Mô tả use case	Người dùng có thể thay đổi thông tin như họ, tên, ngày sinh, giới tính
Actor chính	Người dùng, Quản trị viên
Actor phụ	Không có
Tiền điều kiện	- Có kết nối mạng Internet - Đang đăng nhập hệ thống
Hậu điều kiện	Thêm/xóa/sửa thông tin thành công
Luồng hoạt động	1. Bấm vào ảnh đại diện người dùng 2. Chọn Thông tin cá nhân 3. Thêm/sửa/xóa các thông tin được phép 4. Nhấn chọn xác nhận Cập nhật thông tin
Luồng thay thế	3.1. Thêm/sửa thông tin sai định dạng thì thông báo lỗi
Luồng ngoại lệ	Khi hệ thống gặp sự cố thì thông báo lỗi

Bảng 3.5. Đặc tả use case Quản lý thông tin cá nhân

3.2.2.6. Đổi mật khẩu

Use case ID	UC06_mình
Tên use case	Đổi mật khẩu
Mô tả use case	Cho phép người dùng có thể đổi mật khẩu tài khoản khi có nhu cầu
Actor chính	Người dùng, Quản trị viên

Actor phụ	Không có
Tiền điều kiện	- Có kết nối mạng Internet - Đang đăng nhập hệ thống
Hậu điều kiện	- Đổi mật khẩu thành công
Luồng hoạt động	1. Bấm vào ảnh đại diện của người dùng 2. Chọn Thông tin cá nhân 3. Chọn Đổi mật khẩu 4. Điền mật khẩu hiện tại, mật khẩu mới, xác nhận mật khẩu mới 5. Nhấn chọn xác nhận Đổi mật khẩu
Luồng thay thế	4.1. Nếu mật khẩu hiện tại không đúng thì thông báo lỗi 4.2. Nếu mật khẩu mới và xác nhận mật khẩu mới không trùng khớp thì thông báo lỗi
Luồng ngoại lệ	Khi hệ thống gặp sự cố thì thông báo lỗi

Bảng 3.6. Đặc tả use case Đổi mật khẩu

3.2.2.7. Quản lý dữ liệu chatbot

UCID	UC07_mình
Tên use case	Quản lý dữ liệu chatbot
Mô tả use case	Quản trị viên có thể xem chi tiết hoặc thêm/xóa tài liệu huấn luyện chatbot
Actor chính	Quản trị viên
Actor phụ	Không có
Tiền điều kiện	- Có kết nối mạng Internet - Đang đăng nhập hệ thống
Hậu điều kiện	Thêm/xóa dữ liệu thành công

Luồng hoạt động	1. Quản trị viên vào trang Admin dashboard 2. Chọn datasets -> hiển thị danh sách toàn bộ datasets 3. Chọn nút Thêm datasets 4. Nhập tiêu đề, mô tả và upload tệp dữ liệu từ thiết bị 5. Nhấn nút xác nhận Thêm
Luồng thay thế	4.1. Không nhập thông tin tiêu đề thì sẽ thông báo lỗi 4.2. Upload tệp dữ liệu không phải là pdf hay csv thì sẽ thông báo lỗi
Luồng ngoại lệ	Khi hệ thống gặp sự cố thì thông báo lỗi

Bảng 3.7. Đặc tả use case chức năng Quản lý dữ liệu chatbot

3.2.2.8. Trò chuyện, tư vấn

UCID	UC08_mình
Tên use case	Trò chuyện, tư vấn
Mô tả use case	Cho phép người dùng có thể giao tiếp với chatbot về những thông tin sức khỏe và y tế
Actor chính	Người dùng, Quản trị viên
Actor phụ	Không có
Tiền điều kiện	- Có kết nối mạng Internet - Đã đăng nhập vào hệ thống
Hậu điều kiện	Gửi tin nhắn trò chuyện, tư vấn thành công
Luồng hoạt động	1. Người dùng tạo phiên trò chuyện bằng cách nhấn vào nút Tư vấn mới hoặc chọn một phiên trò chuyện có sẵn 2. Điền nội dung câu hỏi vào khung trò chuyện 3. Nhấn nút gửi 4. Gửi tin nhắn thành công và nhận được phản hồi từ chatbot

Luồng thay thế	2.1. Nội dung câu hỏi trống thì sẽ không nhấn được nút gửi
Luồng ngoại lệ	- Khi hệ thống gặp sự cố thì thông báo lỗi - Khi chatbot không phản hồi thì thông báo lỗi

Bảng 3.8. Đặc tả use case chức năng Trò chuyện, tư vấn

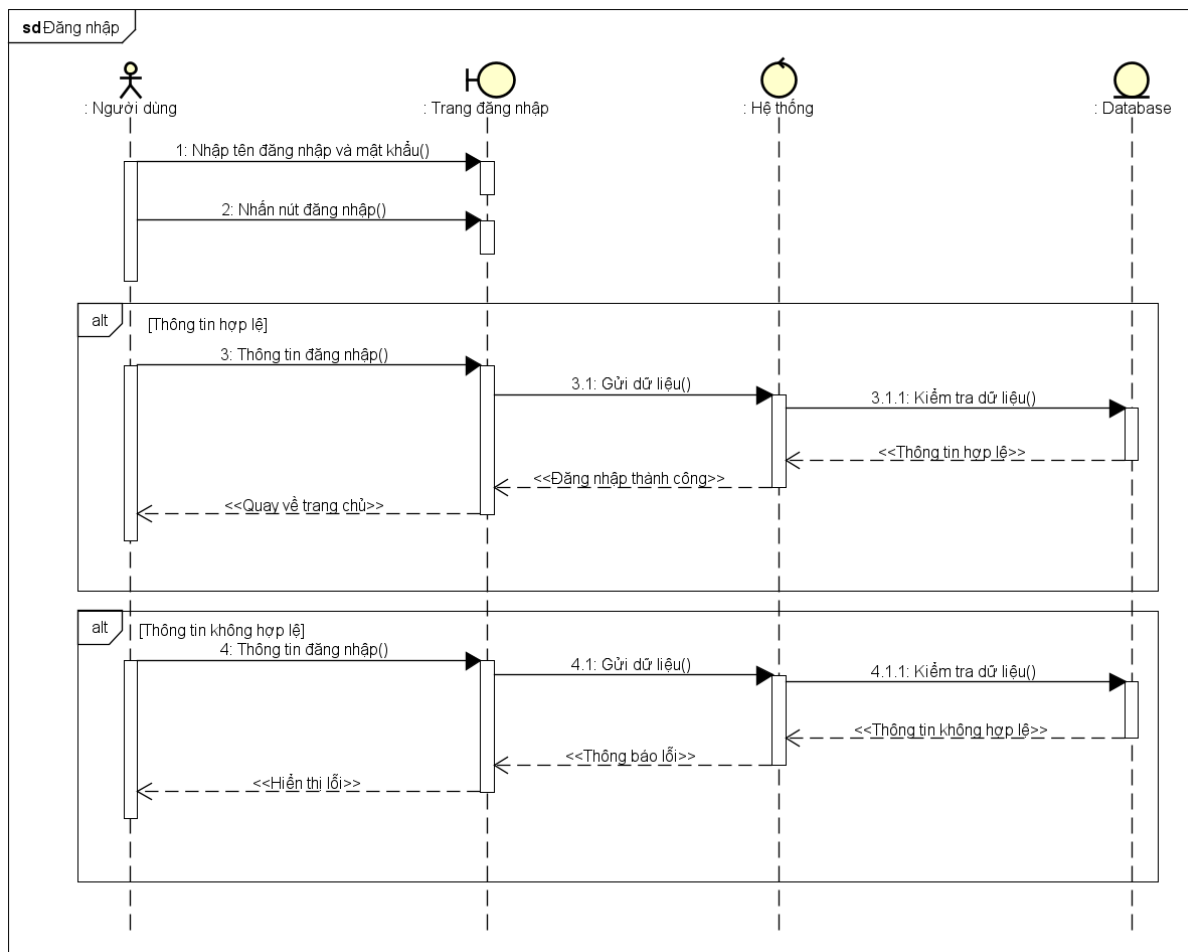
3.2.2.9. Quản lý người dùng

Use case ID	UC09_mình
Tên use case	Quản lý người dùng
Mô tả use case	Quản trị viên có thể quản lý toàn bộ người dùng trong hệ thống
Actor chính	Quản trị viên
Actor phụ	Không có
Tiền điều kiện	- Quản trị viên đăng nhập vào hệ thống - Có kết nối mạng Internet
Hậu điều kiện	Khóa tài khoản thành công
Luồng hoạt động	1. Quản trị viên vào trang Admin dashboard 2. Chọn người dùng -> hiển thị danh sách toàn bộ người dùng 3. Khóa tài khoản người dùng hoặc xem chi tiết người dùng
Luồng thay thế	
Luồng ngoại lệ	Khi hệ thống gặp sự cố thì thông báo lỗi

Bảng 3.9. Đặc tả use case chức năng Quản lý người dùng

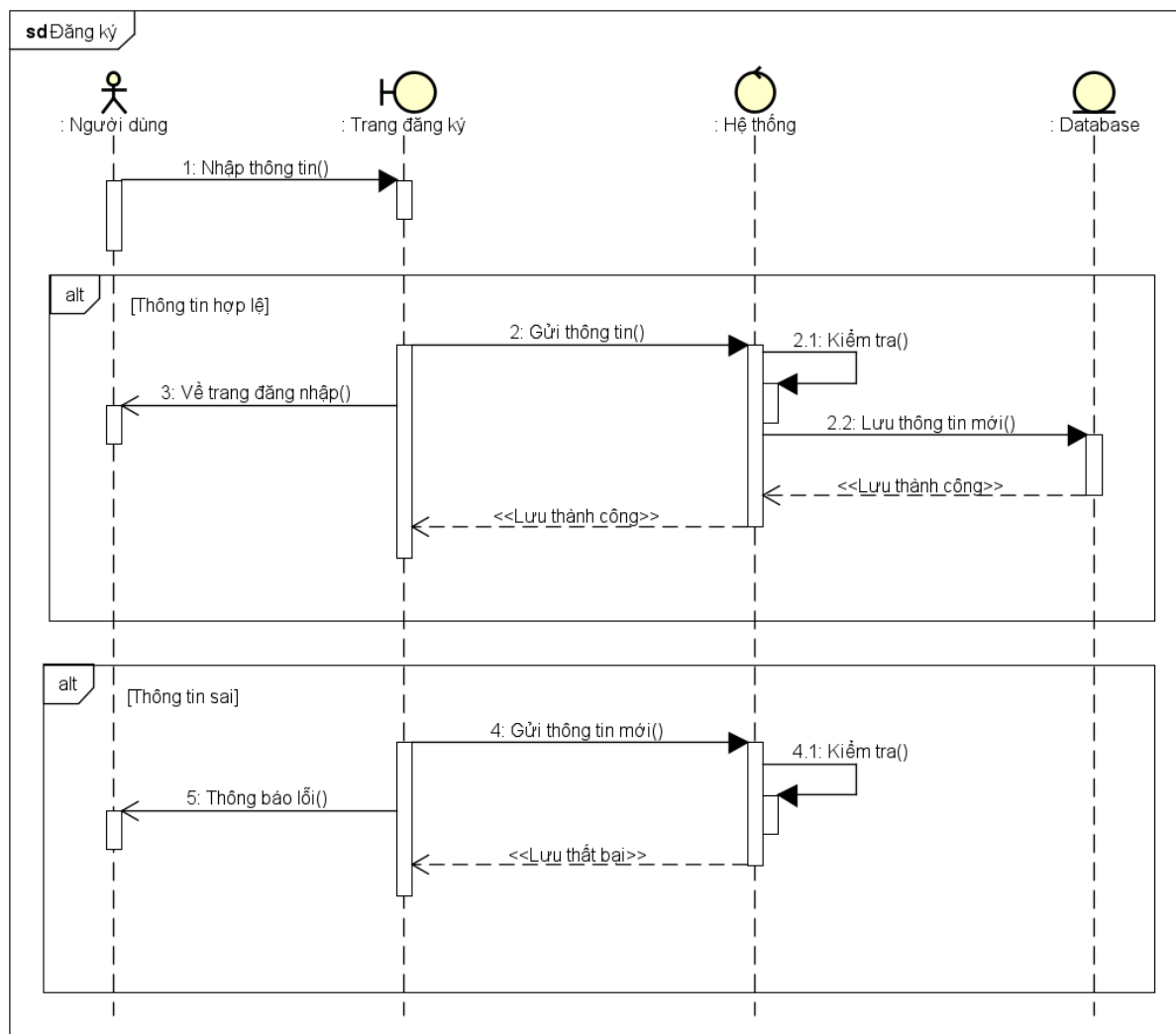
3.2.3. Sơ đồ tuần tự

3.2.3.1. Đăng nhập



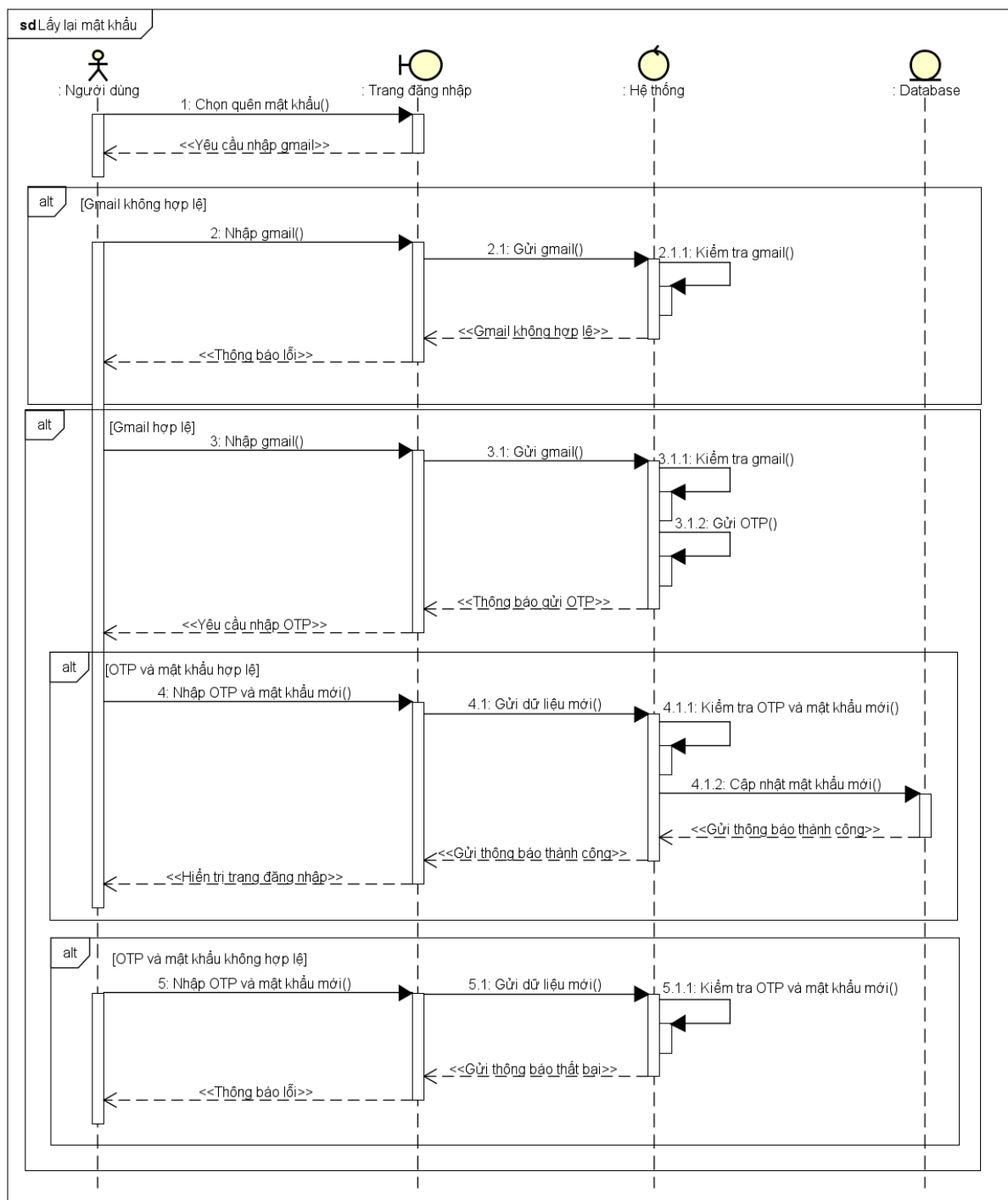
Hình 3.2. Sơ đồ tuần tự chức năng Đăng nhập

3.2.3.2. Đăng ký



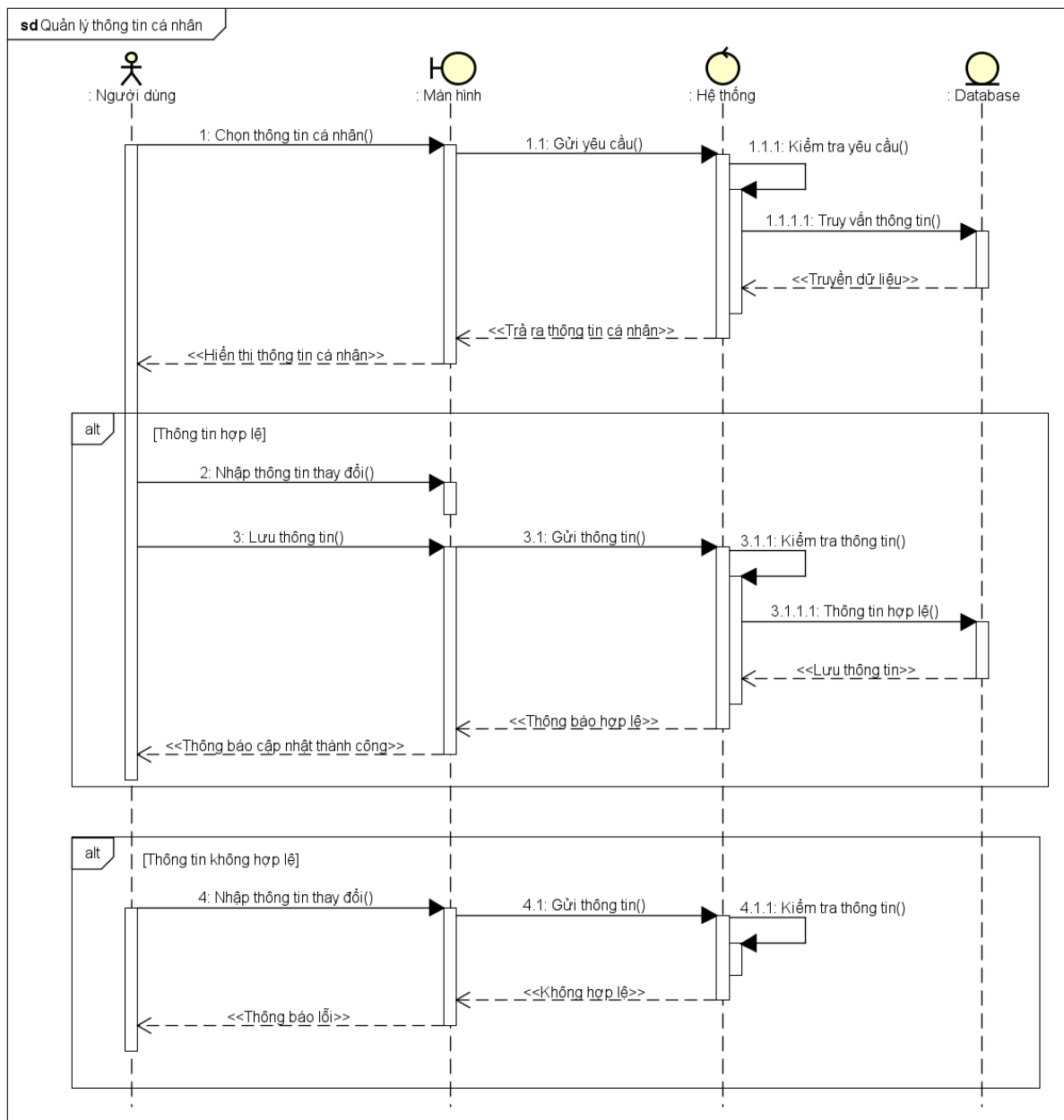
Hình 3.3. Sơ đồ tuần tự chức năng Đăng ký

3.2.3.3. Lấy lại mật khẩu



Hình 3.4. Sơ đồ tuần tự chức năng Lấy lại mật khẩu

3.2.3.4. Quản lý thông tin cá nhân



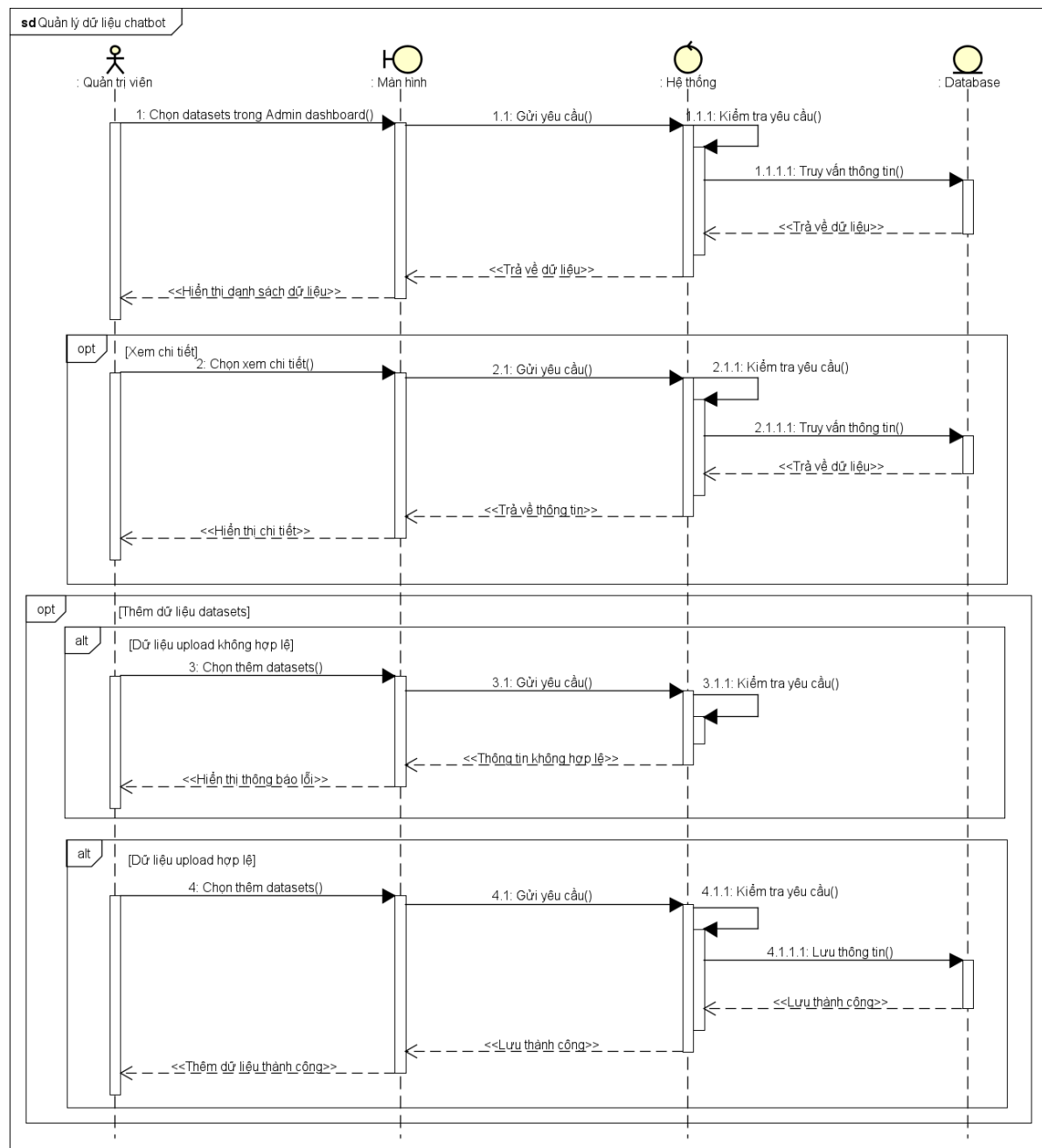
Hình 3.5. Sơ đồ tuần tự chức năng Quản lý thông tin cá nhân

3.2.3.5. Đổi mật khẩu



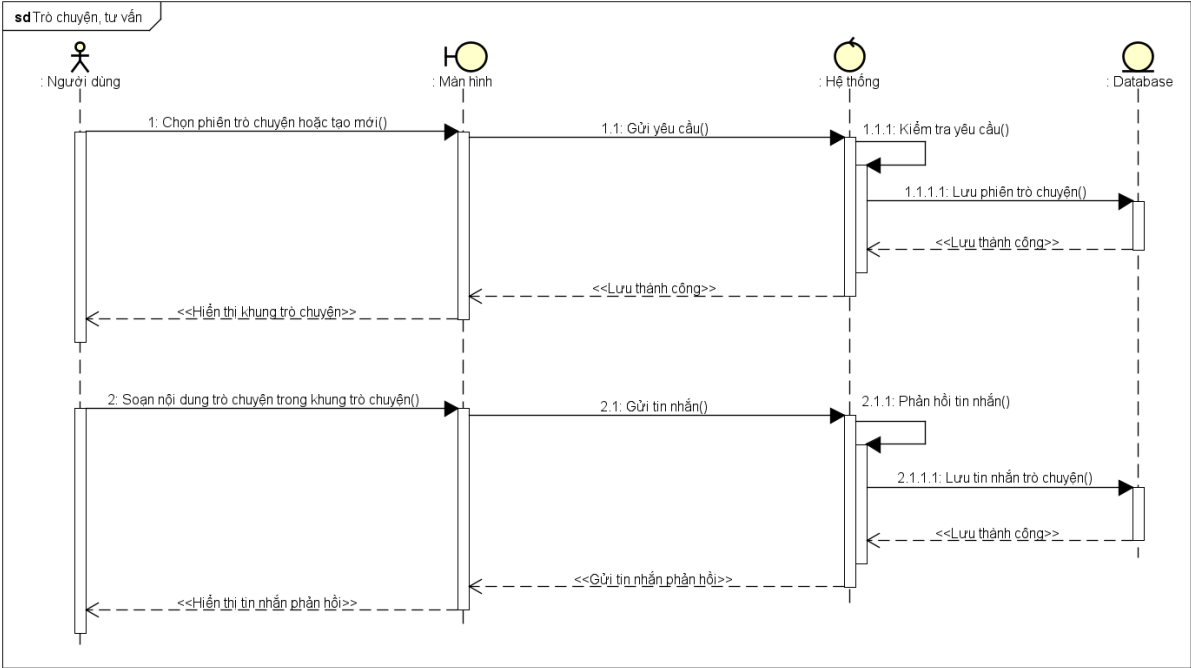
Hình 3.6. Sơ đồ tuần tự chức năng Đổi mật khẩu

3.2.3.6. Quản lý dữ liệu chatbot



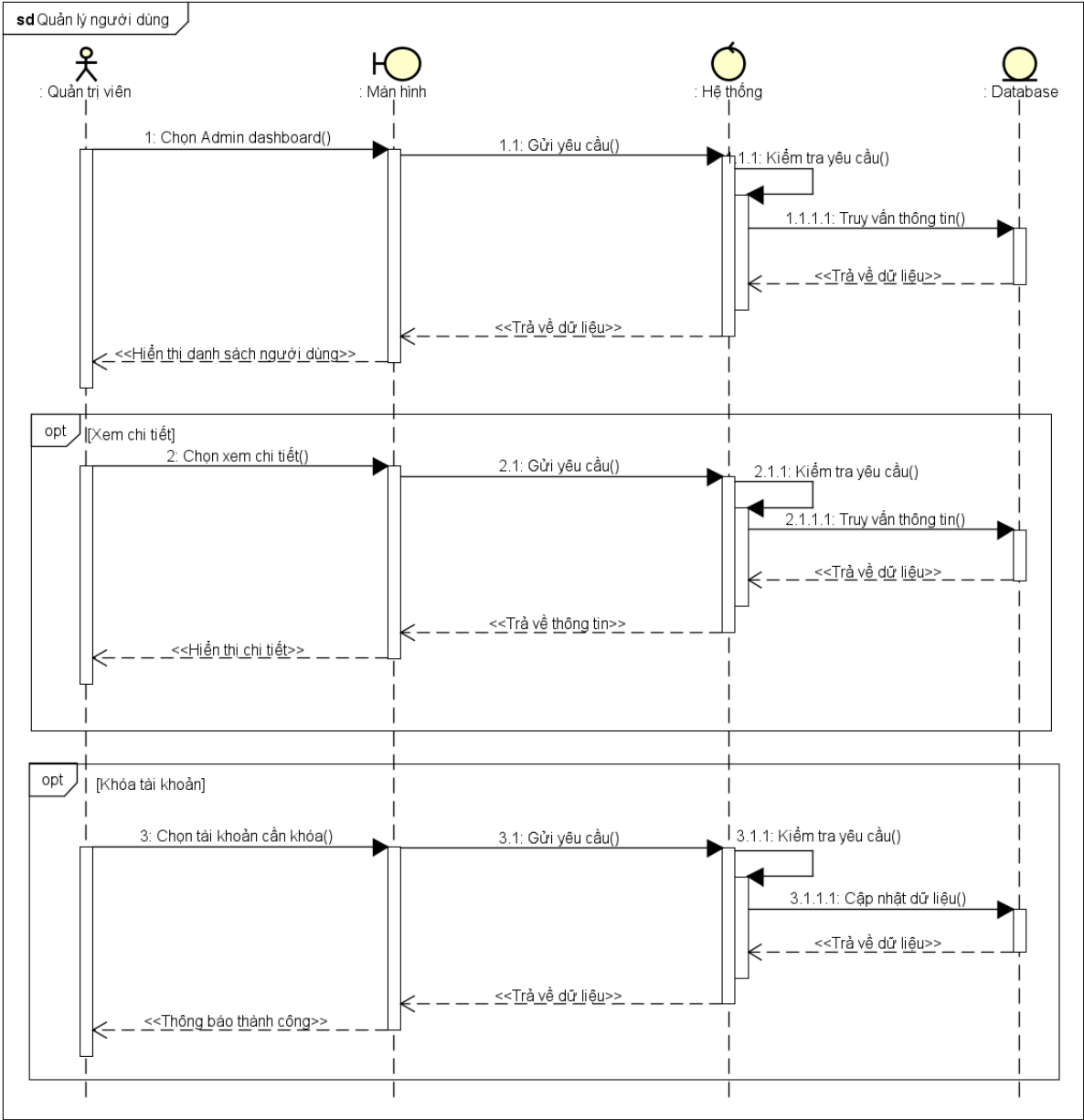
Hình 3.7. Sơ đồ tuần tự chức năng Quản lý dữ liệu chatbot

3.2.3.7. Trò chuyện, tư vấn



Hình 3.8. Sơ đồ tuần tự chức năng Trò chuyện, tư vấn

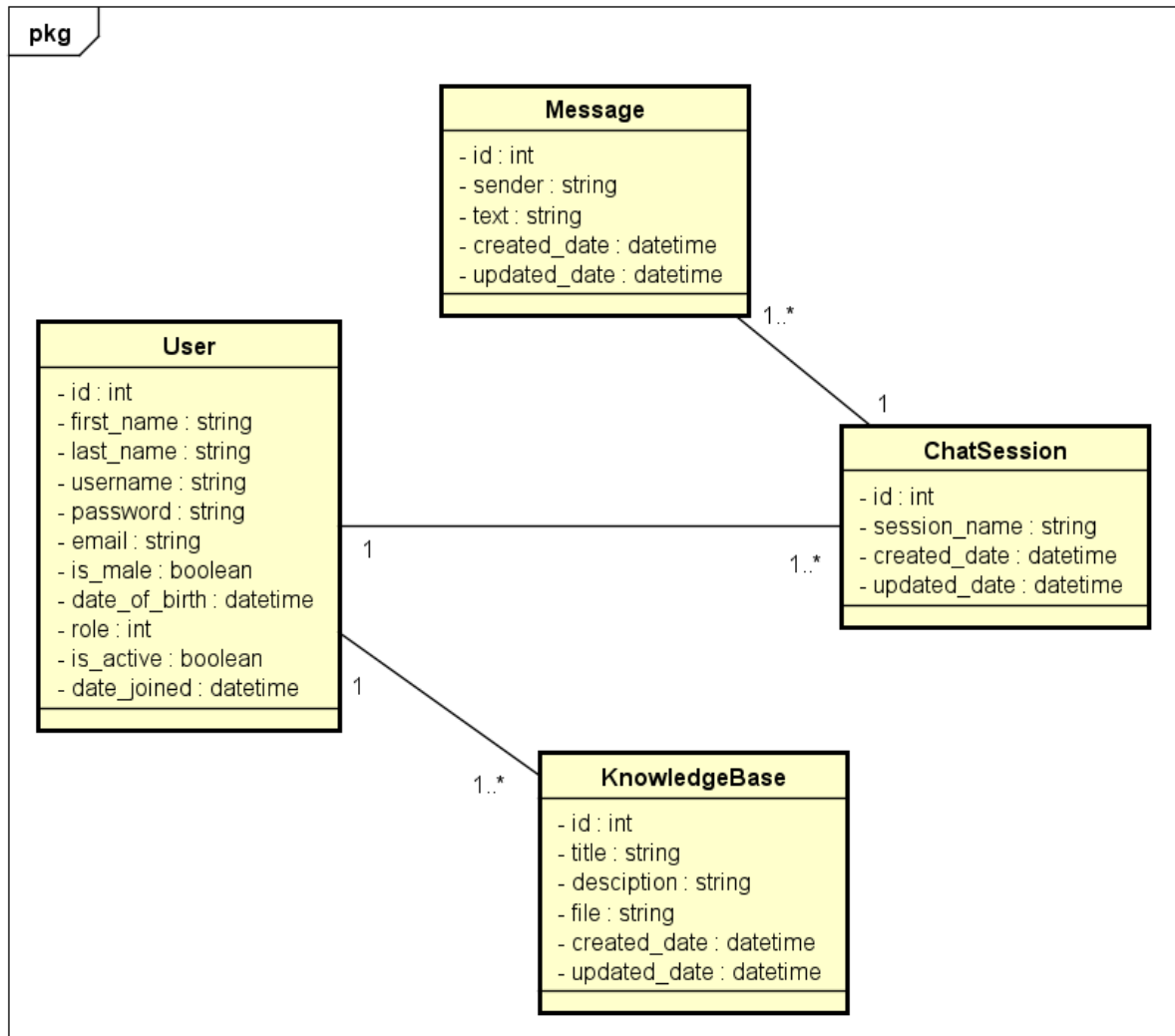
3.2.3.8. Quản lý người dùng



Hình 3.9. Sơ đồ tuần tự chức năng Quản lý người dùng

3.3. Thiết kế hệ thống

3.3.1. Sơ đồ lớp



Hình 3.10. Sơ đồ lớp

Sơ đồ lớp trên mô tả cấu trúc dữ liệu và mối quan hệ giữa các thành phần chính trong hệ thống chatbot. Các lớp đại diện cho những thực thể cốt lõi bao gồm User, ChatSession, Message và KnowledgeBase.

Lớp User:

- Đại diện cho người dùng của hệ thống.
- Các thuộc tính: id, first_name, last_name, username, password, email, is_male, date_of_birth, role, is_active, date_joined.
- Thông tin này giúp quản lý danh tính, phân quyền, và trạng thái hoạt động của người dùng.

Lớp ChatSession:

- Đại diện cho một phiên trò chuyện của người dùng với chatbot.
- Các thuộc tính: id, session_name, created_date, updated_date.
- Một người dùng có thể có nhiều phiên trò chuyện khác nhau.

Lớp Message:

- Lưu trữ từng tin nhắn trong một phiên trò chuyện.
- Các thuộc tính: id, sender, text, created_date, updated_date.

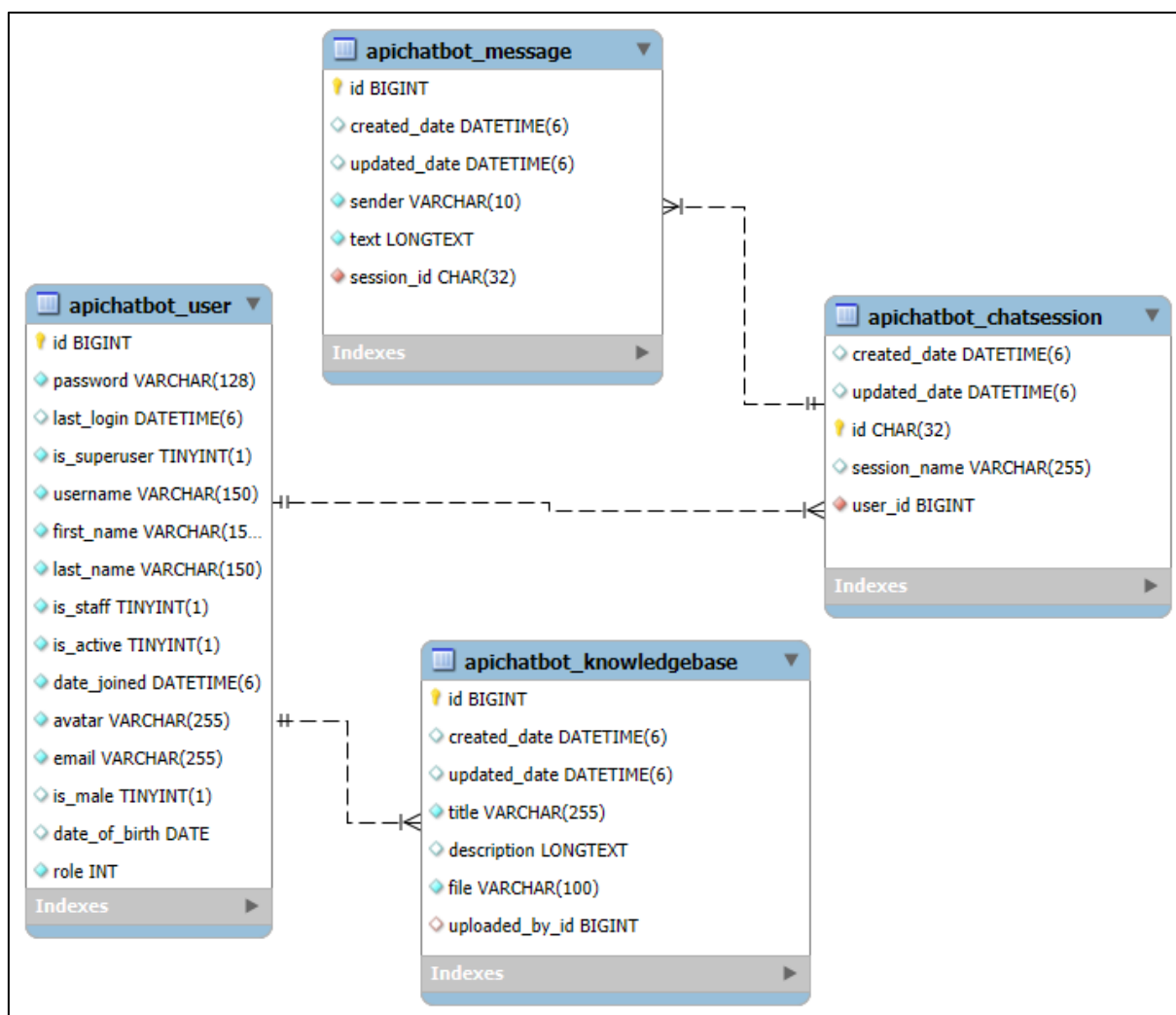
Lớp KnowledgeBase:

- Đại diện cho kho dữ liệu tri thức mà chatbot sử dụng để trả lời người dùng.
- Các thuộc tính: id, title, description, file, created_date, updated_date.

Sơ đồ lớp này cung cấp một cái nhìn tổng quan về cách các đối tượng và mối quan hệ của chúng được tổ chức trong hệ thống, giúp dễ dàng hiểu được cấu trúc của dữ liệu và các quy tắc trong hệ thống này.

3.3.2. Thiết kế cơ sở dữ liệu

3.3.2.1. Lược đồ cơ sở dữ liệu



Hình 3.11. Lược đồ cơ sở dữ liệu của hệ thống

Lược đồ được thiết kế dựa trên các thực thể cốt lõi gồm User, ChatSession, Message và KnowledgeBase, cùng với mối quan hệ giữa chúng. Thiết kế này đảm bảo dữ liệu được tổ chức chặt chẽ, dễ mở rộng và hỗ trợ tốt cho các chức năng chính của hệ thống như quản lý người dùng, lưu trữ hội thoại, cũng như quản lý tri thức y tế cho chatbot.

3.3.2.2. Các bảng dữ liệu của hệ thống

STT	Tên trường	Kiểu dữ liệu	Diễn giải	Khuôn dạng	PK	FK
1	id	INT	Mã người dùng	Số	x	
2	username	VARCHAR(150)	Tên tài khoản	Ký tự		

3	password	VARCHAR(128)	Mật khẩu	Ký tự		
4	last_login	DATETIME(6)	Lần đăng nhập cuối	dd/MM/yyyy		
5	is_superuser	BOOLEAN	Là superuser	Số boolean		
6	first_name	VARCHAR(150)	Tên người dùng	Ký tự		
7	last_name	VARCHAR(150)	Họ người dùng	Ký tự		
8	is_staff	BOOLEAN	Là staff	Số boolean		
9	is_active	BOOLEAN	Đang hoạt động	Số boolean		
10	date_joined	DATETIME(6)	Ngày tham gia	dd/MM/yyyy		
11	avatar	VARCHAR(255)	Ảnh đại diện	Ký tự		
12	email	VARCHAR(255)	Email người dùng	Ký tự		
13	is_male	BOOLEAN	Là giới tính nam	Số boolean		
14	date_of_birth	DATE	Ngày sinh	dd/MM/yyyy		
15	role	INT	Vai trò	Số		

Bảng 3.10. user

STT	Tên trường	Kiểu dữ liệu	Diễn giải	Khuôn dạng	PK	FK
1	id	CHAR(32)	Mã phiên trò chuyện	Ký tự	x	
2	session_name	VARCHAR(255)	Tên phiên trò chuyện	Ký tự		
3	created_date	DATETIME(6)	Ngày tạo	dd/MM/yyyy		
4	updated_date	DATETIME(6)	Ngày chỉnh sửa	dd/MM/yyyy		
5	user_id	INT	Mã người	Số		x

			dùng			
--	--	--	------	--	--	--

Bảng 3.11. chatsession

STT	Tên trường	Kiểu dữ liệu	Diễn giải	Khuôn dạng	PK	FK
1	id	INT	Mã tin nhắn	Số	x	
2	sender	VARCHAR(10)	Người gửi	Ký tự		
3	text	VARCHAR(255)	Nội dung tin nhắn	Ký tự		
4	created_date	DATETIME(6)	Ngày tạo	dd/MM/yyyy		
5	updated_date	DATETIME(6)	Ngày chỉnh sửa	dd/MM/yyyy		
6	session_id	CHAR(32)	Mã phiên trò chuyện	Ký tự		x

Bảng 3.12. message

STT	Tên trường	Kiểu dữ liệu	Diễn giải	Khuôn dạng	PK	FK
1	id	INT	Mã dữ liệu	Số	x	
2	title	VARCHAR(255)	Tiêu đề	Ký tự		
3	description	VARCHAR(255)	Mô tả	Ký tự		
4	file	VARCHAR(100)	Dữ liệu	Ký tự		
5	created_date	DATETIME(6)	Ngày tạo	dd/MM/yyyy		
6	updated_date	DATETIME(6)	Ngày chỉnh sửa	dd/MM/yyyy		
7	upload_by_id	INT	Mã người dùng	Số		x

Bảng 3.13. knowledgebase

3.3.2.3. Môi quan hệ của từng bảng

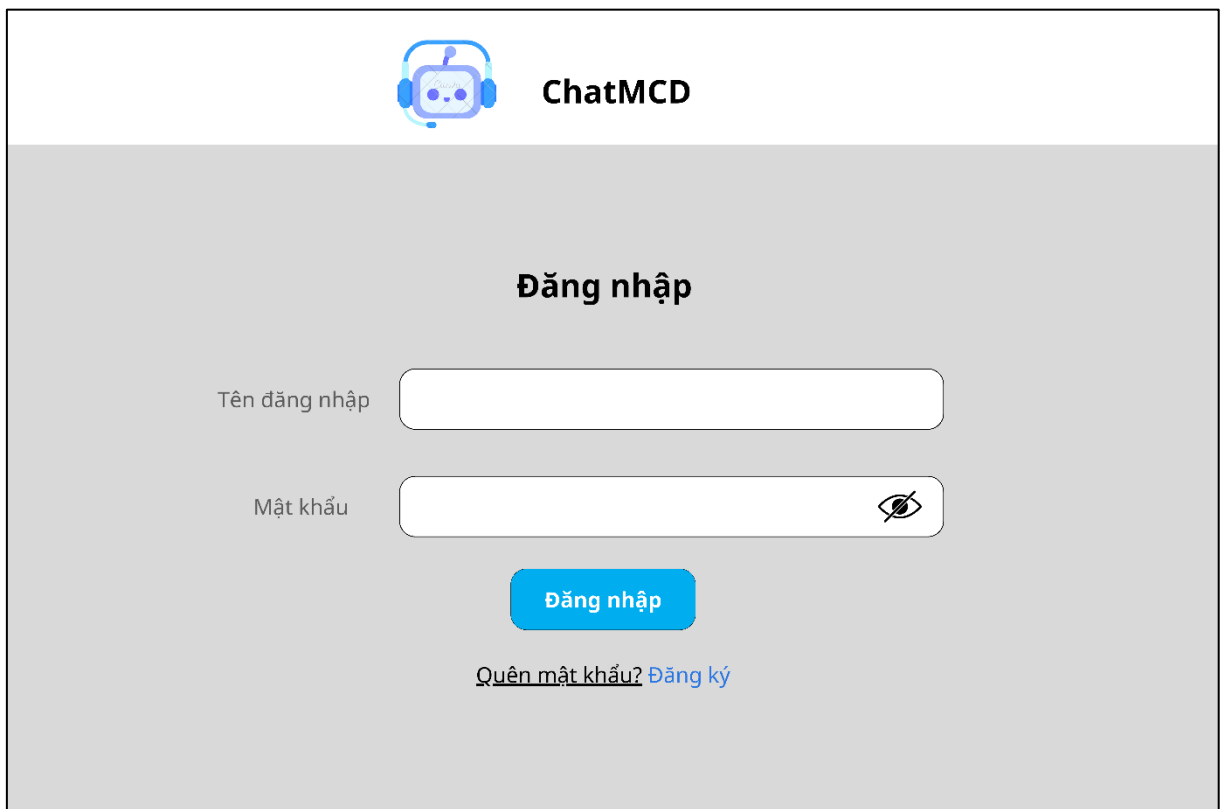
User – ChatSession: quan hệ 1 – n, một người dùng có thể tạo nhiều phiên trò chuyện, một phiên trò chuyện chỉ thuộc về một người dùng.

ChatSession – Message: quan hệ 1 – n, một phiên trò chuyện thì có nhiều tin nhắn trò chuyện, một tin nhắn trò chuyện chỉ thuộc về một phiên trò chuyện.

User – KnowledgeBase: quan hệ 1 – n, một người dùng có thể tạo nhiều dữ liệu học, một dữ liệu học chỉ được tạo bởi một người dùng.

3.3.3. Thiết kế giao diện

3.3.3.1. Đăng nhập



ChatMCD

Đăng nhập

Tên đăng nhập

Mật khẩu 

[Đăng nhập](#)

[Quên mật khẩu?](#) [Đăng ký](#)

Hình 3.12. Giao diện Đăng nhập

3.3.3.2. Đăng ký

 ChatMCD

Đăng ký

Họ

Tên

Tên đăng nhập

Email

Ngày sinh

Giới tính

Avatar

Mật khẩu

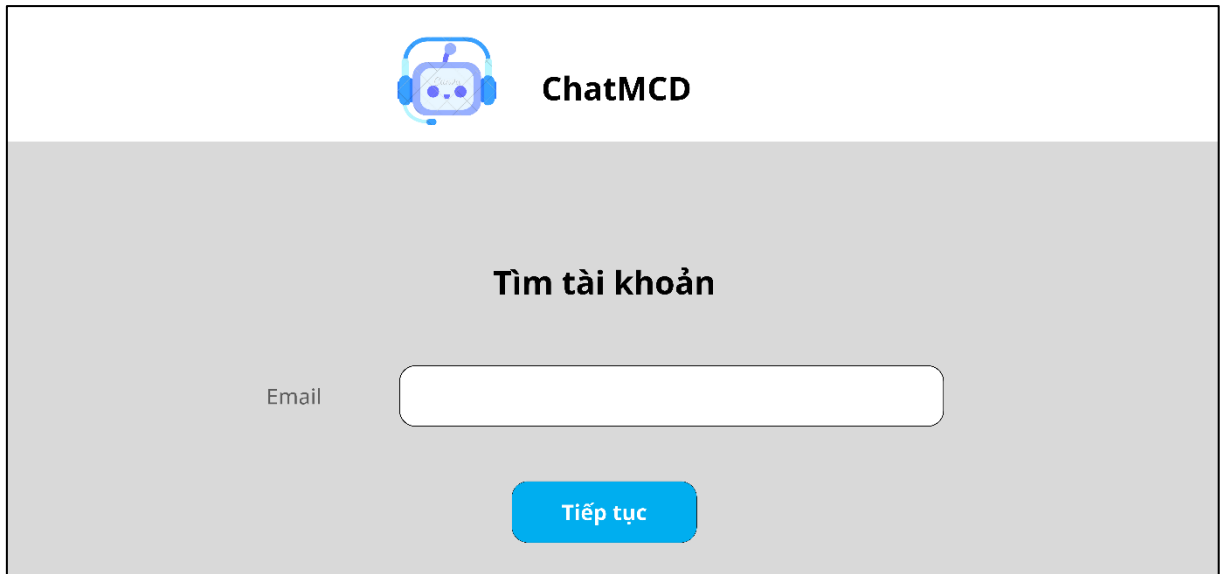
Nhập lại mật khẩu

Tạo tài khoản

Đã có tài khoản? [Đăng nhập](#)

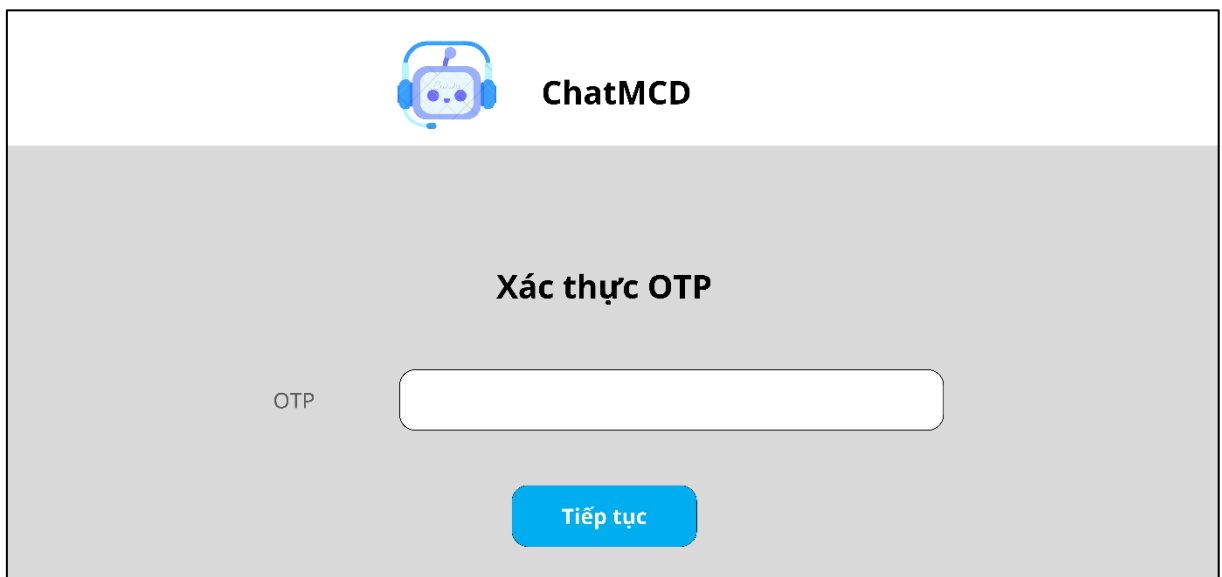
Hình 3.13. Giao diện Đăng ký

3.3.3.3. Lấy lại mật khẩu



The screenshot shows the 'ChatMCD' interface for finding an account. At the top, there is a logo of a blue robot head with a microphone and the text 'ChatMCD'. Below this, the title 'Tìm tài khoản' (Find account) is centered. Under the title, the label 'Email' is positioned to the left of a white rectangular input field. Below the input field, there is a blue button with the white text 'Tiếp tục' (Continue).

Hình 3.14. Giao diện kiểm tra Email



The screenshot shows the 'ChatMCD' interface for verifying an OTP. At the top, there is a logo of a blue robot head with a microphone and the text 'ChatMCD'. Below this, the title 'Xác thực OTP' (Verify OTP) is centered. Under the title, the label 'OTP' is positioned to the left of a white rectangular input field. Below the input field, there is a blue button with the white text 'Tiếp tục' (Continue).

Hình 3.15. Giao diện xác thực OTP



ChatMCD

Đổi mật khẩu

Mật khẩu

Nhập lại mật khẩu

Tiếp tục

Hình 3.16. Giao diện Đổi mật khẩu mới

3.3.3.4. Trò chuyện, tư vấn



ChatMCD

Tư vấn mới

Danh sách phiên trò chuyện

-
-
-
-

Nội dung



Hình 3.17. Giao diện Trò chuyện, tư vấn

3.3.3.5. Quản lý thông tin cá nhân

 ChatMCD

Thông tin cá nhân

Họ

Tên

Email

Ngày sinh

Giới tính



Tên đăng nhập: username

Cập nhật thông tin

Hình 3.18. Giao diện Quản lý thông tin cá nhân

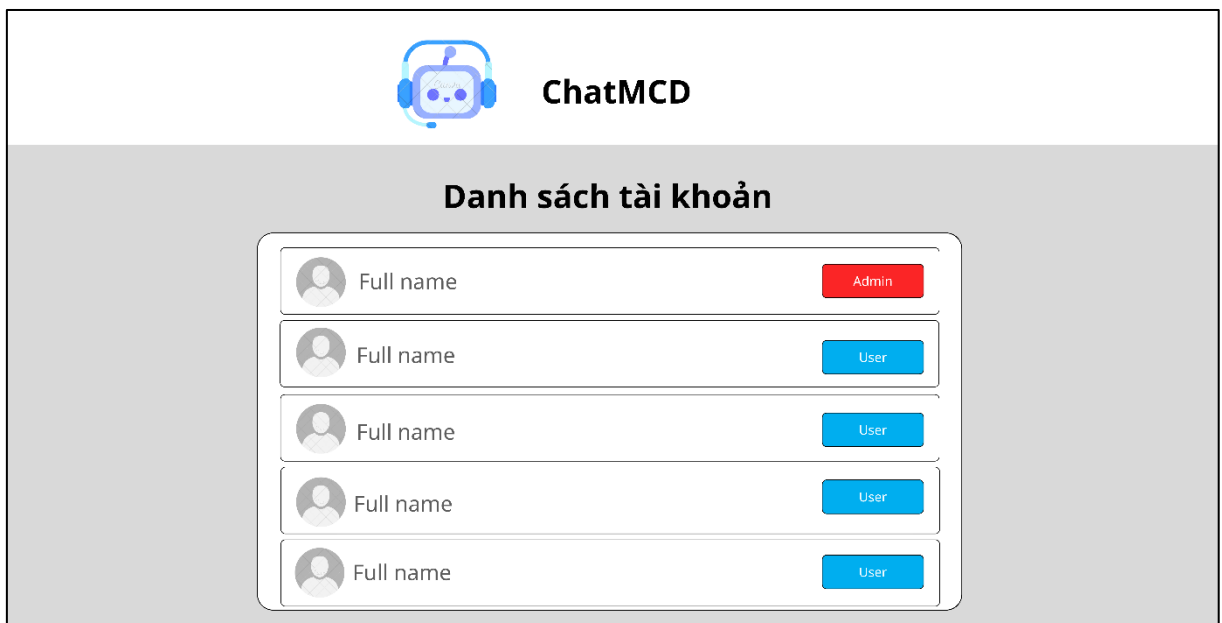
3.3.3.6. Đổi mật khẩu



The screenshot shows the 'ChatMCD' application header with a robot icon. Below the header, the title 'Đổi mật khẩu' (Change Password) is centered. The form contains three input fields: 'Mật khẩu hiện tại' (Current Password), 'Mật khẩu' (New Password), and 'Nhập lại mật khẩu' (Repeat New Password). Each field has a toggle icon (an eye with a slash) to the right. At the bottom, there is a blue button labeled 'Xác nhận' (Confirm).

Hình 3.19. Giao diện Đổi mật khẩu

3.3.3.7. Quản lý người dùng

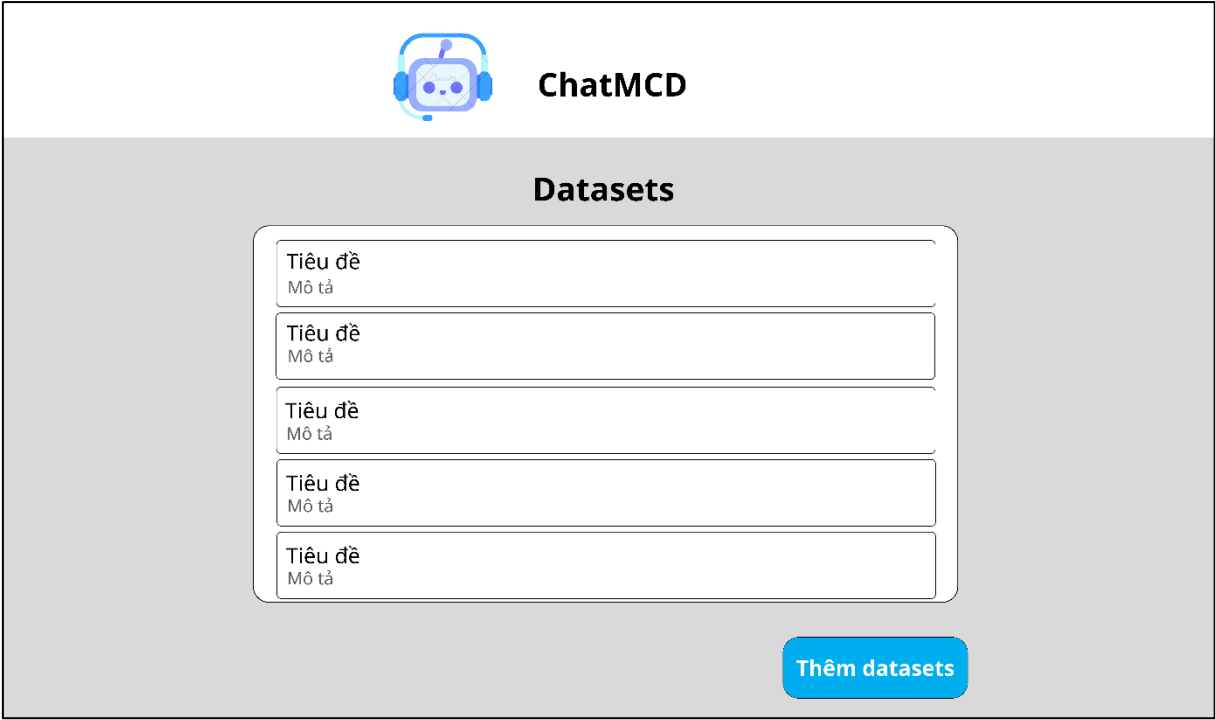


The screenshot shows the 'ChatMCD' application header with a robot icon. Below the header, the title 'Danh sách tài khoản' (Account List) is centered. The main content area displays a list of five user accounts, each with a placeholder profile picture, the text 'Full name', and a role button. The first account has a red 'Admin' button, while the other four have blue 'User' buttons.

Full name	Role
Full name	Admin
Full name	User
Full name	User
Full name	User
Full name	User

Hình 3.20. Giao diện Quản lý người dùng

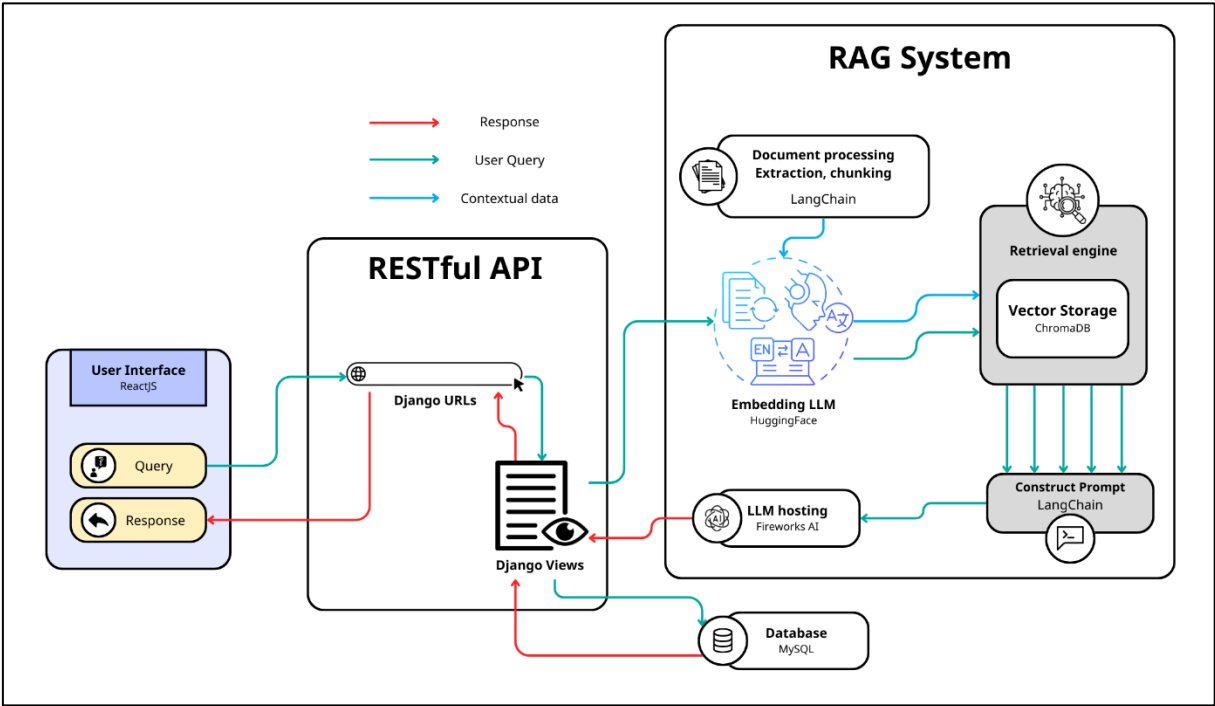
3.3.3.8. Quản lý dữ liệu chatbot



Hình 3.21. Giao diện Quản lý dữ liệu chatbot

3.4. Kiến trúc hệ thống

3.4.1. Sơ đồ kiến trúc hệ thống



Hình 3.22. Sơ đồ kiến trúc hệ thống

3.4.2. Các thành phần trong kiến trúc

User Interface (ReactJS):

- Vai trò: Đây là giao diện frontend nơi người dùng tương tác với hệ thống, cho phép người dùng truy vấn thông tin (query) và nhận phản hồi (response).
- Cách hoạt động: Sử dụng ReactJS để render giao diện, quản lý trạng thái (state) và gửi yêu cầu HTTP đến RESTful API. Sau khi backend xử lý, response được hiển thị trên giao diện người dùng (dưới dạng văn bản, danh sách hoặc các yếu tố tương tác).

RESTful API:

- Vai trò: Là lớp trung gian giữa frontend và backend, xử lý các yêu cầu từ người dùng, định tuyến chúng đến các chức năng phù hợp trong Django Views và trả về kết quả.
- Cách hoạt động: Django URLs ánh xạ các đường dẫn URL đến các hàm xử lý cụ thể trong Django Views để xử lý logic nghiệp vụ, gọi đến RAG System để xử lý truy vấn và trả về kết quả dưới dạng JSON hoặc HTML. Ngoài ra, Django Views có thể tích hợp với database để lưu trữ và truy xuất dữ liệu.

Document Processing (LangChain):

- Vai trò: Các tài liệu được chuẩn bị và nạp vào hệ thống để sử dụng trong retrieval bằng cách thực hiện tách nhỏ văn bản (chunking) để phù hợp với mô hình ngôn ngữ. Kết quả của việc làm này giúp tài liệu được chuẩn hóa, sẵn sàng cho bước embedding.
- Cách hoạt động: Trích xuất nội dung (extraction) từ các nguồn tài liệu, sử dụng các công cụ để lấy văn bản sạch, loại bỏ các hình ảnh hay định dạng không cần thiết. Sau đó chia tài liệu lớn thành các phần nhỏ (chunks) để dễ quản lý và embedding, mỗi chunks thường có kích thước cố định để tránh vượt quá giới hạn của mô hình. Sử dụng LangChain để điều phối quy trình từ extraction đến embedding.

Embedding LLM (HuggingFace):

- Vai trò: Chuyển đổi văn bản đã được chunks từ Document Processing thành vector embeddings, đây là biểu diễn số học để đo lường sự tương đồng nghĩa (semantic).

- Cách hoạt động: Sử dụng mô hình từ HuggingFace như Sentence Transformers hoặc BERT-based models để tạo vector. Các vector đại diện cho ý nghĩa của văn bản và cho phép so sánh cosine similarity.

Vector Storage (ChromaDB):

- Vai trò: Lưu trữ và truy xuất các chunks tài liệu liên quan nhất với truy vấn của người dùng. Đóng vai trò như “trí nhớ dài hạn” của hệ thống.
- Cách hoạt động: ChromaDB - một vector database nhẹ, open-source, dùng để lưu trữ và index embeddings. Nó hỗ trợ tìm kiếm nhanh bằng các thuật toán như HNSW. Khi nhận query, embed query thành vector, sau đó query ChromaDB để tìm top-k chunks có similarity cao nhất. Retrieval Engine điều phối quá trình này có thể kết hợp với reranking để cải thiện độ chính xác.

LLM Hosting (Fireworks AI):

- Vai trò: Lưu trữ và chạy mô hình ngôn ngữ lớn để sinh ra response dựa trên prompt.
- Cách hoạt động: Fireworks AI là một nền tảng cloud để host LLM như Llama hoặc GPT-like, hỗ trợ API để gọi mô hình, xử lý inference với tốc độ cao, hỗ trợ scaling. LLM nhận prompt từ Construct Prompt và sinh ra câu trả lời tự nhiên.

Construct Prompt (LangChain):

- Vai trò: Xây dựng prompt đầy đủ bằng cách kết hợp truy vấn, dữ liệu ngữ cảnh (contextual data), và các chunks retrieved để gửi đến LLM.
- Cách hoạt động: Sử dụng LangChain để template prompt, đảm bảo LLM không tạo thông tin giả (hallucinate) mà dựa vào dữ liệu thực. Sau khi lấy ra các đoạn văn bản liên quan, hệ thống sẽ ghép chúng với Prompt Template định hướng cho LLM cách trả lời. Đây là bước quan trọng để đảm bảo chất lượng và an toàn y tế.

Database (MySQL):

- Vai trò: Lưu trữ dữ liệu cấu trúc, như metadata của tài liệu, lịch sử truy vấn, hoặc thông tin người dùng.

- Cách hoạt động: Là một RDBMS open-source, MySQL sử dụng SQL để quản lý bảng như user, chat session, document (metadata), hỗ trợ Django Views trong việc lưu/truy xuất dữ liệu.

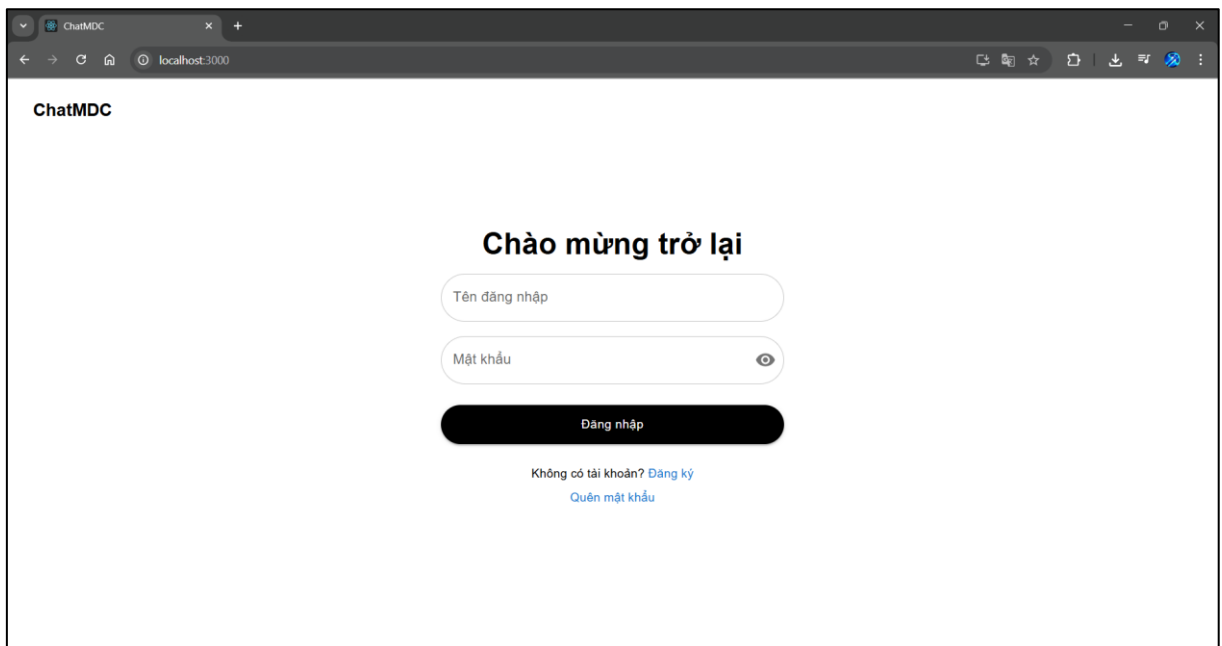
3.4.3. Cơ chế hoạt động của kiến trúc

Kiến trúc hệ thống hoạt động theo cơ chế như sau: Người dùng thực thi truy vấn từ giao diện ReactJS, sau đó yêu cầu này được gửi đến RESTful API do Django đảm nhiệm. Tại đây, Django URLs định tuyến request tới Django Views, nơi đóng vai trò điều phối vừa kết nối với cơ sở dữ liệu MySQL để quản lý thông tin người dùng và phiên trò chuyện vừa chuyển câu hỏi đến hệ thống RAG. Trong RAG System, dữ liệu tri thức được xử lý trước bằng LangChain (trích xuất, chia nhỏ và mã hoá thành vector thông qua Embedding LLM của HuggingFace), sau đó lưu trữ trong ChromaDB để hỗ trợ tìm kiếm ngữ nghĩa. Khi có truy vấn mới, hệ thống tiến hành so khớp trong Vector Storage để tìm ra các đoạn văn bản phù hợp, kết hợp cùng câu hỏi người dùng để xây dựng prompt hoàn chỉnh bằng LangChain. Prompt này được gửi đến mô hình ngôn ngữ lớn (LLM) được host trên Fireworks AI để sinh phản hồi. Cuối cùng, câu trả lời sẽ được Django trả về frontend và hiển thị trực tiếp trên giao diện của người dùng.

3.5. Kết quả đạt được

3.5.1. Chức năng Đăng nhập

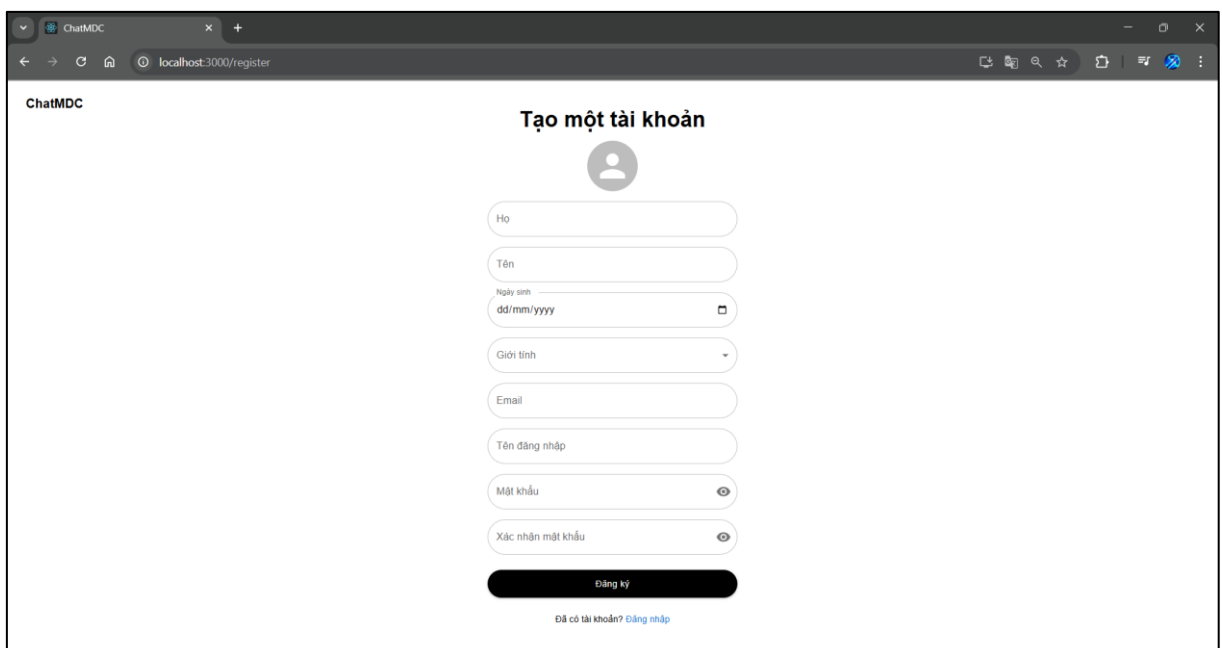
Người dùng thực hiện đăng nhập bằng tài khoản đã được đăng ký trong hệ thống.



Hình 3.23. Giao diện chức năng đăng nhập

3.5.2. Chức năng Đăng ký

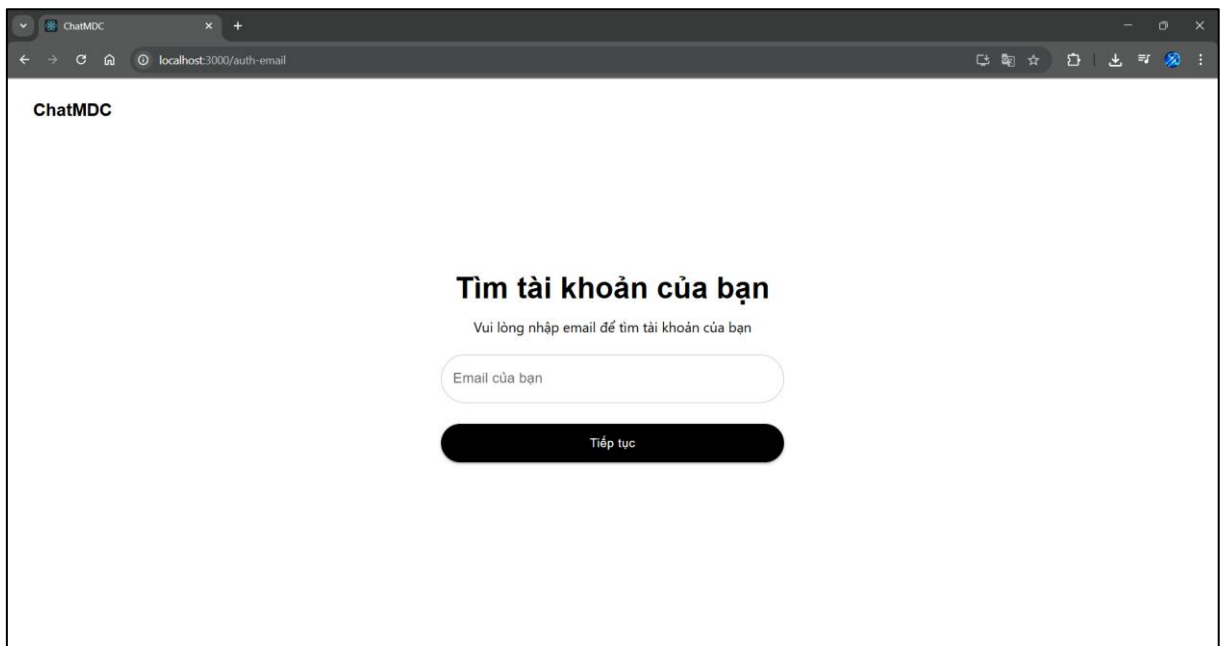
Nếu người dùng chưa có tài khoản thì sẽ nhấn đăng ký để tạo một tài khoản mới.



Hình 3.24. Giao diện chức năng đăng ký

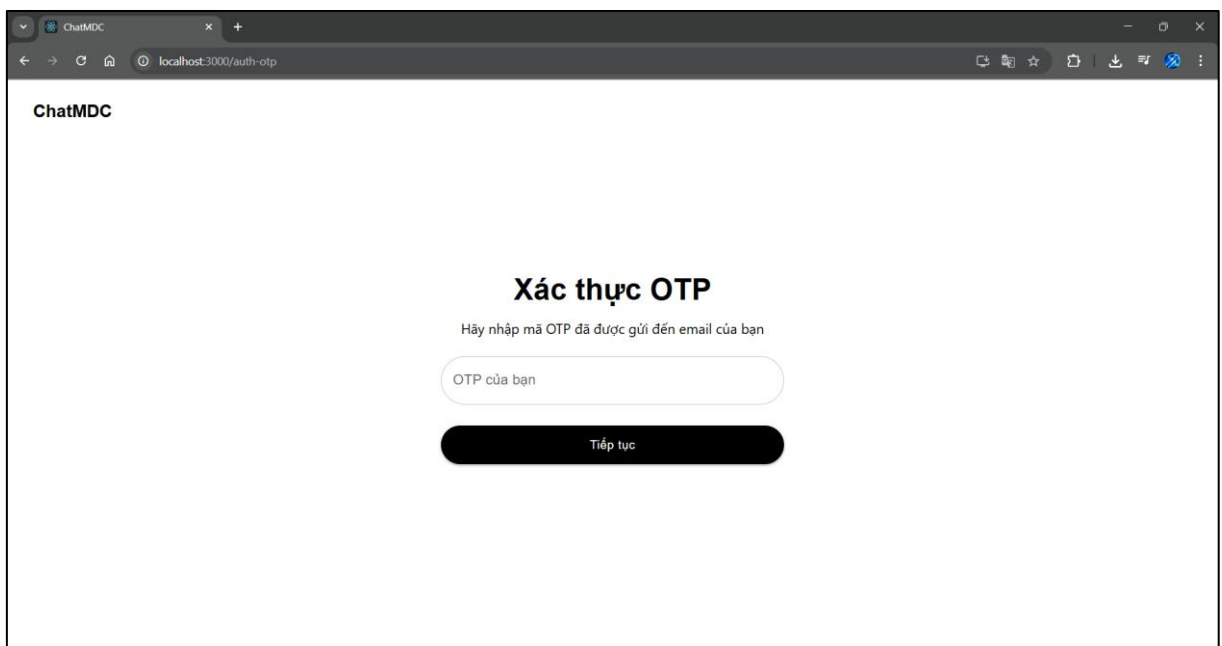
3.5.3. Chức năng Lấy lại mật khẩu

Nếu người dùng quên mật khẩu thì có thể đặt lại mật khẩu bằng Email của tài khoản đó.



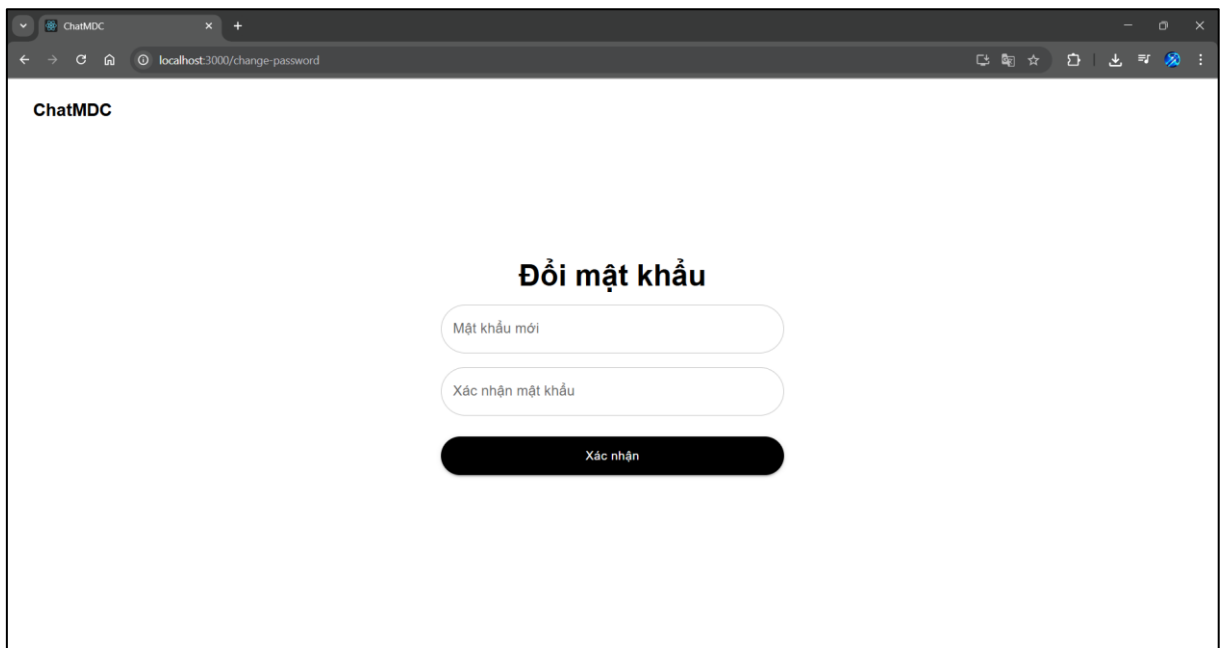
Hình 3.25. Giao diện tìm kiếm tài khoản bằng Email

Người dùng thực hiện xác thực mã OTP đã được gửi qua Email trước đó.



Hình 3.26. Giao diện xác thực mã OTP

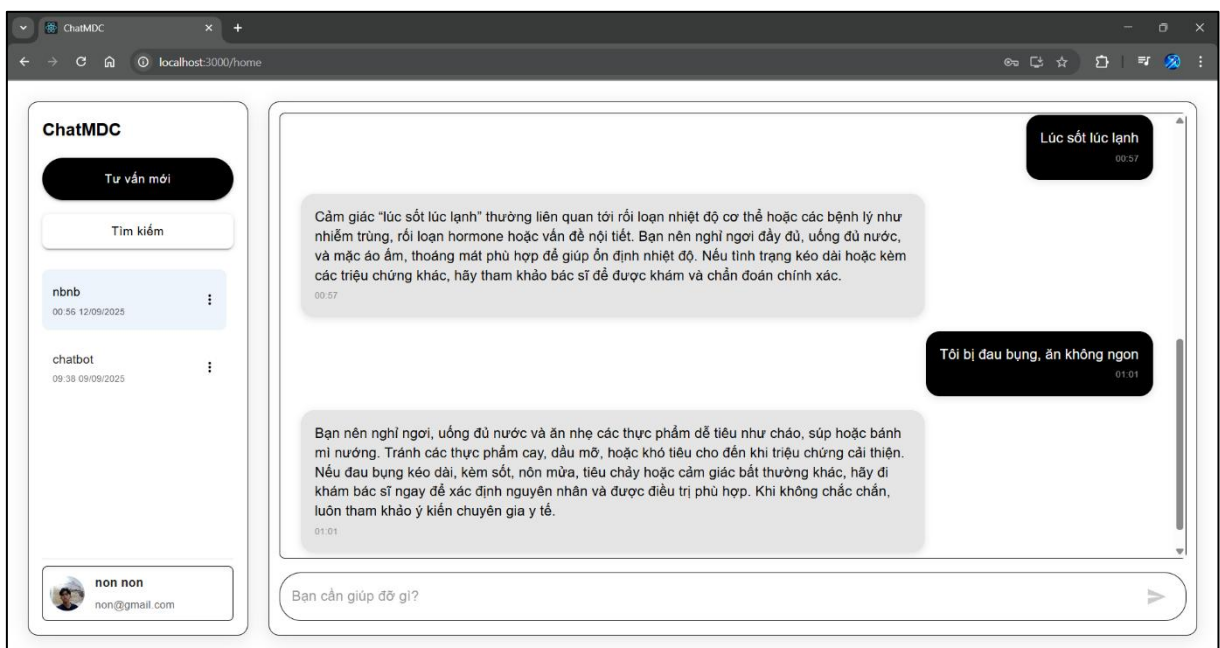
Người dùng thực hiện đổi mật khẩu của tài khoản.



Hình 3.27. Giao diện đặt lại mật khẩu tài khoản

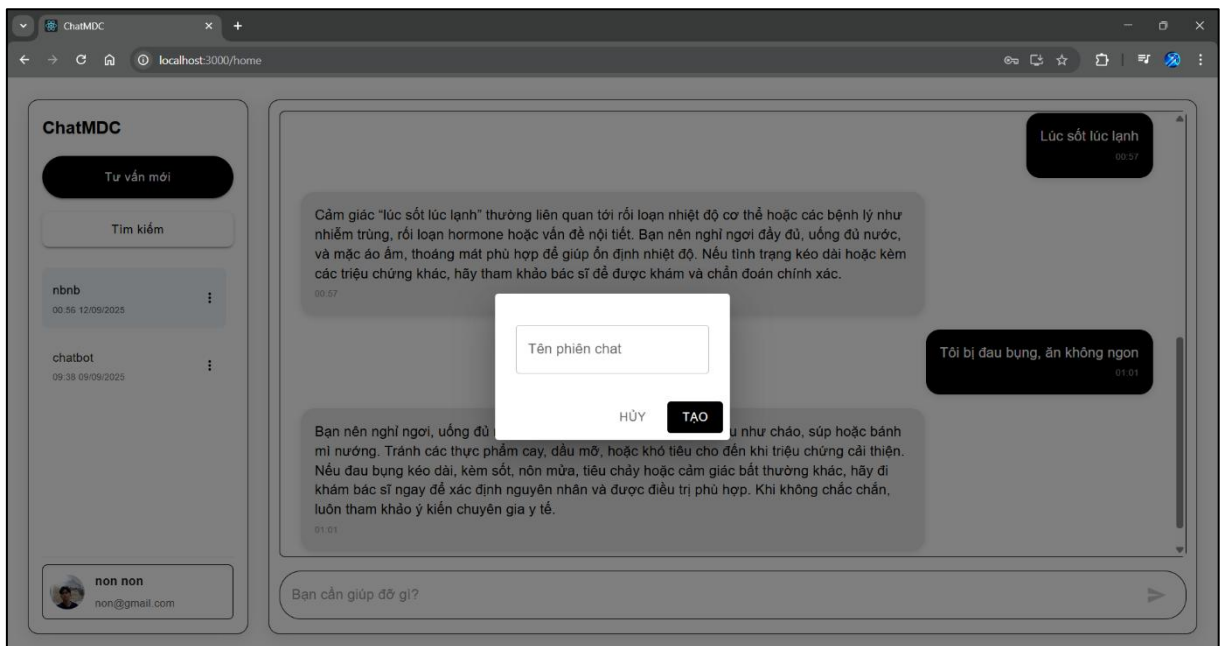
3.5.4. Chức năng Trò chuyện, tư vấn

Người dùng sử dụng chatbot để tư vấn y tế, sức khỏe bằng cách gửi tin nhắn vào một phiên trò chuyện.



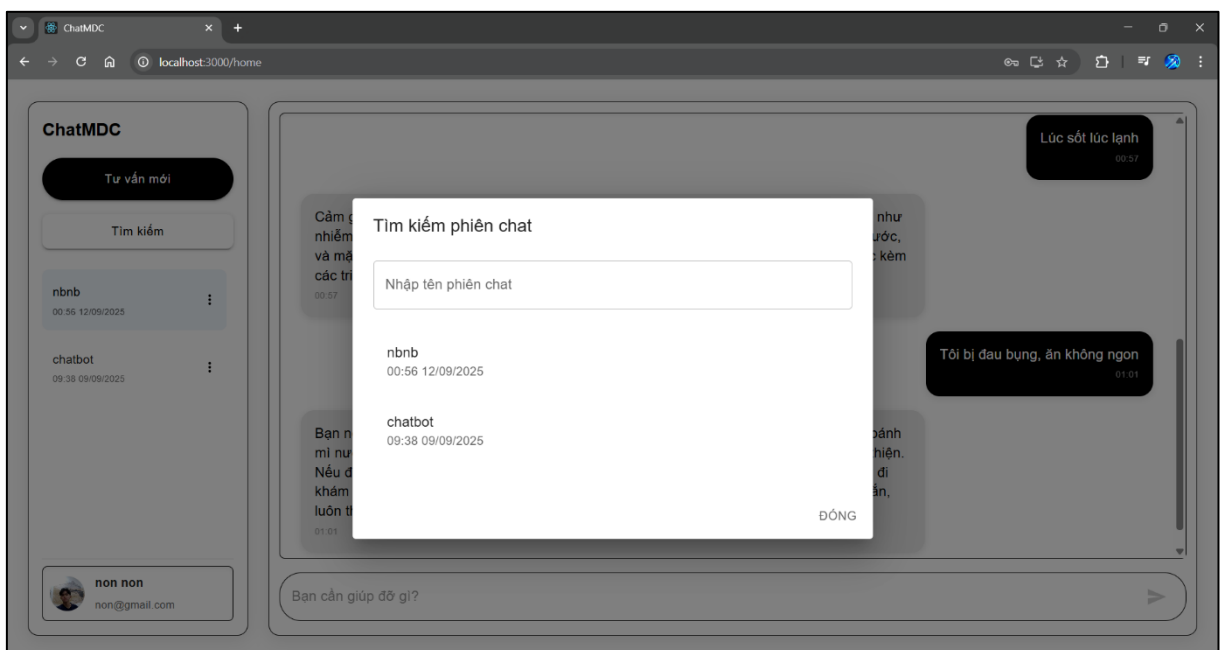
Hình 3.28. Giao diện chức năng trò chuyện, tư vấn

Người dùng có thể tạo mới một phiên trò chuyện khác.



Hình 3.29. Giao diện tạo thêm phiên trò chuyện

Người dùng có thể tìm kiếm phiên trò chuyện đã lưu của mình.



Hình 3.30. Giao diện tìm kiếm phiên trò chuyện

3.5.5. Chức năng Quản lý thông tin cá nhân

Người dùng có thể xem và hoặc thêm/sửa/xóa thông tin cá nhân của mình.

ChatMDC

Thông tin người dùng

[THÔNG TIN](#) [ĐỔI MẬT KHẨU](#)


admin
Tên tài khoản: minh

Họ

Tên

Ngày sinh

Giới tính

Email

[Cập nhật thông tin](#)

Hình 3.31. Giao diện quản lý thông tin cá nhân

3.5.6. Chức năng Đổi mật khẩu

Người dùng có thể chủ động thay đổi mật khẩu của tài khoản khi có nhu cầu.

ChatMDC

Thông tin người dùng

[THÔNG TIN](#) [ĐỔI MẬT KHẨU](#)

Mật khẩu hiện tại

Mật khẩu mới

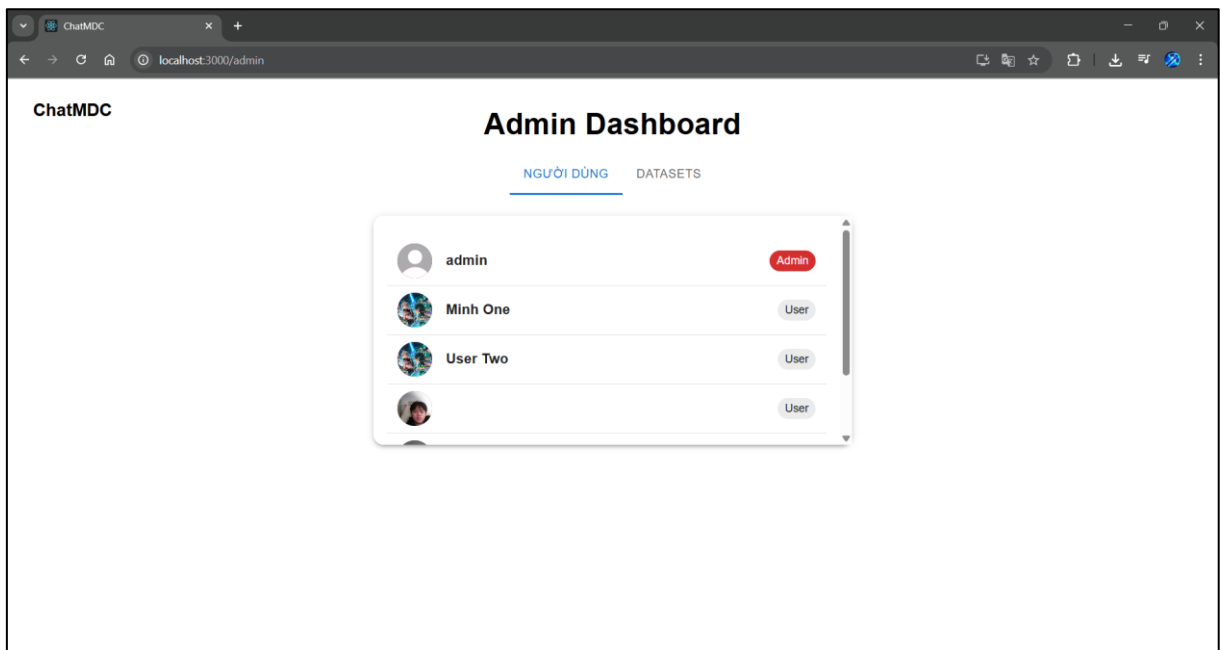
Xác nhận mật khẩu

[Đổi mật khẩu](#)

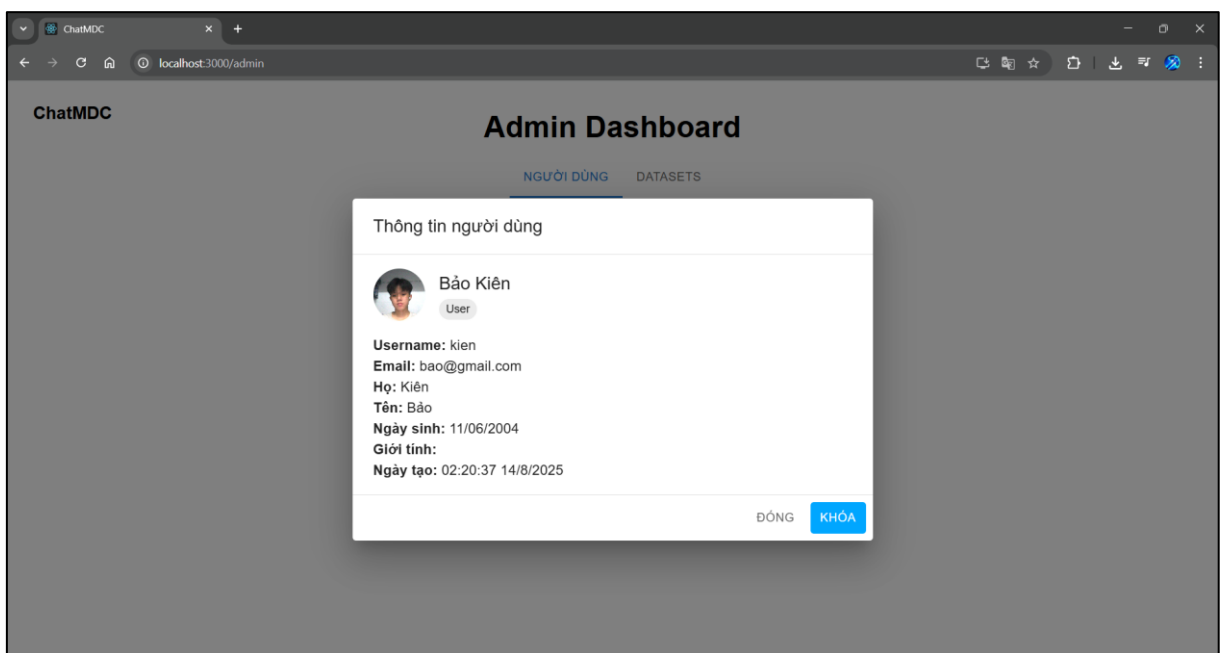
Hình 3.32. Giao diện thay đổi mật khẩu tài khoản

3.5.7. Chức năng Quản lý người dùng

Quản trị viên có thể xem chi tiết hoặc khóa các tài khoản có trong hệ thống.



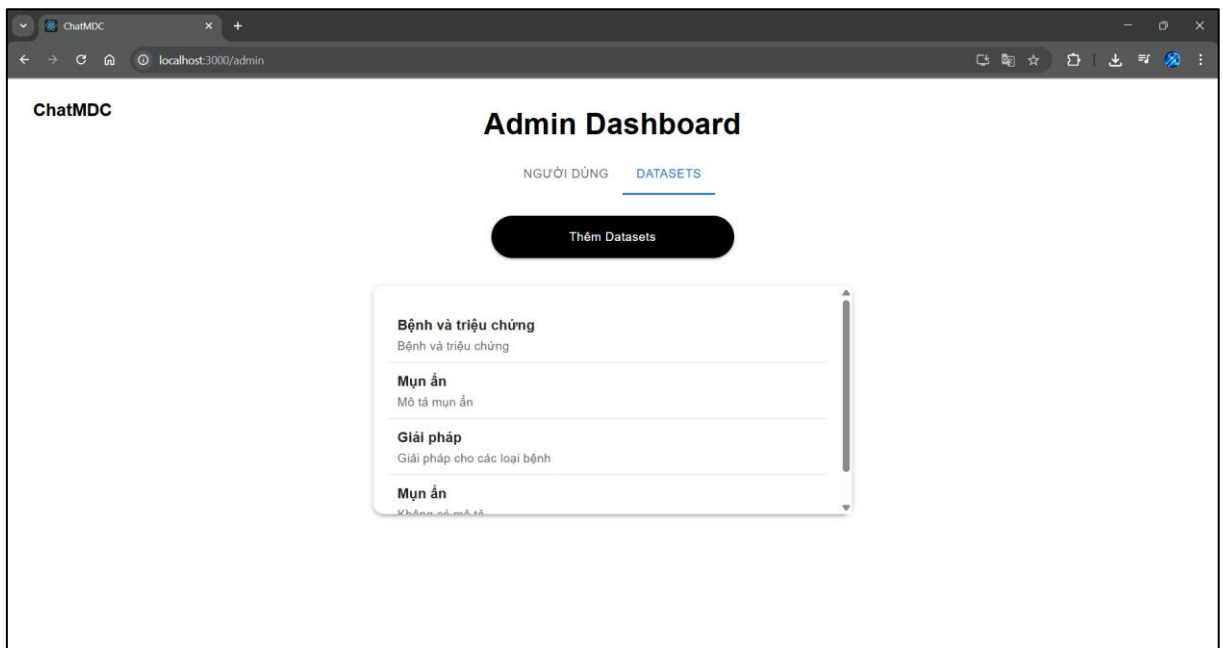
Hình 3.33. Giao diện quản lý tài khoản trong hệ thống



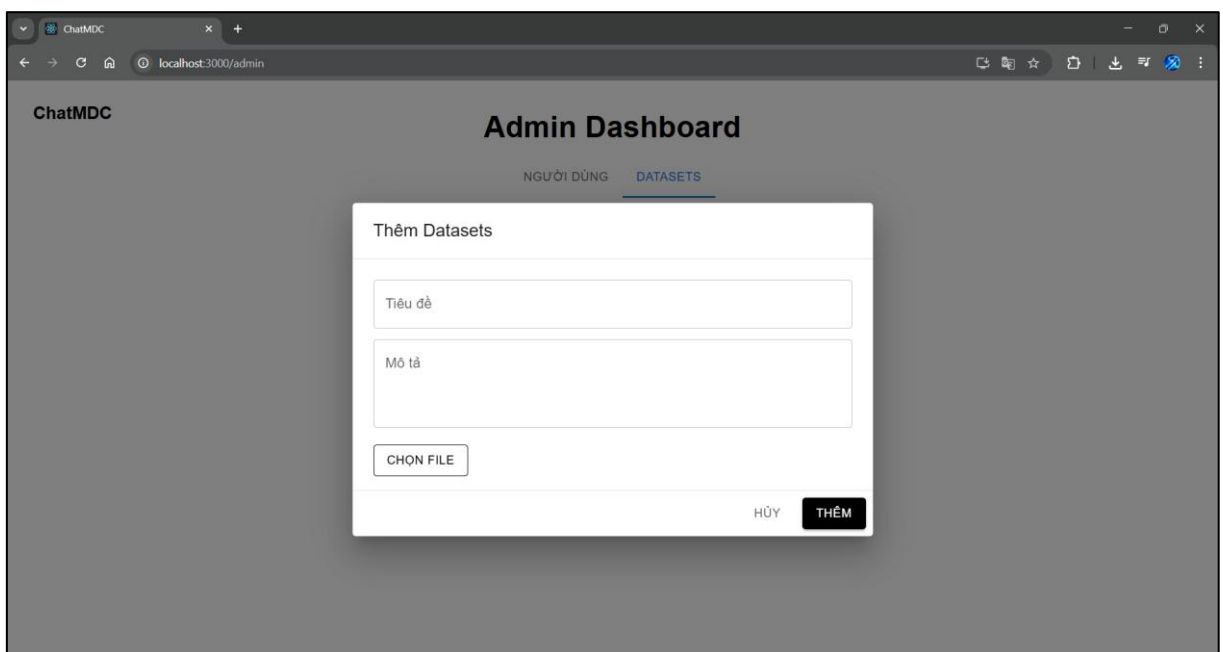
Hình 3.34. Giao diện xem chi tiết tài khoản

3.5.8. Chức năng Quản lý dữ liệu chatbot

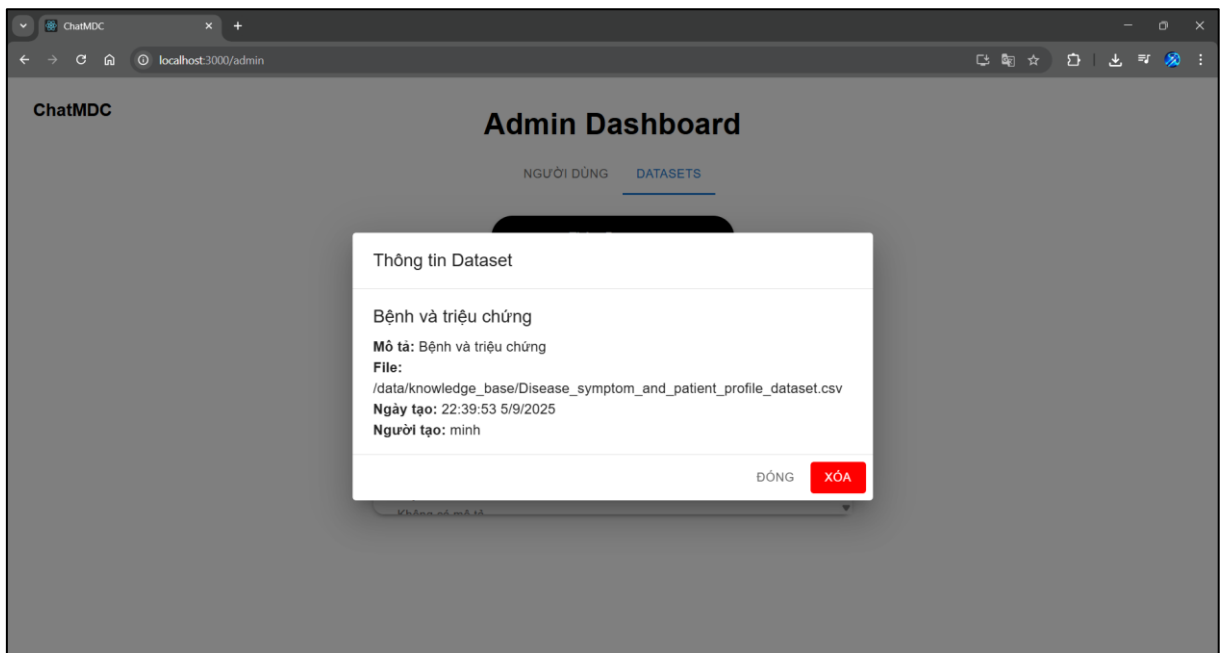
Quản trị viên có thể thêm, xem chi tiết và xóa dữ liệu học cho chatbot.



Hình 3.35. Giao diện quản lý dữ liệu học của chatbot



Hình 3.36. Giao diện thêm dữ liệu học cho chatbot



Hình 3.37. Giao diện xem chi tiết dữ liệu học cho chatbot

Chương 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết luận

Đồ án chatbot tư vấn y tế đã được thực hiện thành công với mục tiêu chính là xây dựng một hệ thống hỗ trợ người dùng tra cứu và tham khảo thông tin sức khỏe một cách nhanh chóng, an toàn và đáng tin cậy. So với các mục tiêu đề ra ban đầu, hệ thống đã đạt được hầu hết các yêu cầu bao gồm việc tích hợp mô hình RAG kết hợp với mô hình ngôn ngữ lớn LLM để cung cấp câu trả lời chính xác dựa trên cơ sở dữ liệu y khoa. Các chức năng cốt lõi như giao diện trò chuyện thân thiện, quản lý lịch sử hội thoại, và quyền quản trị để cập nhật dữ liệu đã được triển khai ổn định, mang lại trải nghiệm tốt cho người dùng. Hệ thống còn đảm bảo tính an toàn bằng cách nhấn mạnh các phản hồi chỉ mang tính tham khảo, không thể thay thế chẩn đoán chuyên môn từ bác sĩ.

Đồ án đã giải quyết được các vấn đề chính như khó khăn trong việc tiếp cận thông tin y tế đáng tin cậy, đặc biệt ở khu vực nông thôn hoặc với những người bận rộn, bằng cách cung cấp một công cụ miễn phí, dễ sử dụng dựa trên AI. Kết quả thực tế cho thấy chatbot có khả năng xử lý câu hỏi đa dạng, trả lời ngắn gọn và dựa trên dữ liệu đã được chọn lọc, giúp giảm tải cho hệ thống y tế truyền thống. Tuy nhiên, một số hạn chế vẫn tồn tại, chẳng hạn như phụ thuộc vào chất lượng dữ liệu đầu vào và khả năng xử lý các câu hỏi phức tạp chưa hoàn hảo.

Về ưu điểm, hệ thống nổi bật với tính linh hoạt nhờ sử dụng LangChain và ChromaDB để quản lý dữ liệu vector, kết hợp với giao diện ReactJS thân thiện và backend Django mạnh mẽ, đảm bảo bảo mật và dễ mở rộng. Chatbot hỗ trợ lịch sử hội thoại giúp cuộc trò chuyện tự nhiên hơn và tích hợp các biện pháp an toàn cho người dùng.

Về khuyết điểm, bao gồm việc mô hình có thể trả lời không chính xác nếu dữ liệu không đầy đủ hoặc cập nhật, chưa hỗ trợ xử lý hình ảnh y tế (ảnh chụp bệnh lý) và hiệu suất có thể bị ảnh hưởng bởi tốc độ API từ Fireworks AI.

4.2. Hướng phát triển

Đồ án đã hoàn thành được các mục tiêu cơ bản nên vẫn còn nhiều tiềm năng để phát triển nhằm nâng cao hiệu quả và mở rộng ứng dụng, một số hướng phát triển có thể được triển khai trong tương lai:

- Tích hợp voice chat sử dụng công nghệ nhận diện giọng nói như Google Speech-to-Text để hỗ trợ người dùng cao tuổi hoặc người khuyết tật.
- Mở rộng cơ sở dữ liệu bằng cách kết nối với nguồn y tế thời gian thực như API từ WHO hoặc Bộ Y tế Việt Nam để cập nhật thông tin về bệnh mới nhất.
- Có thể thêm tính năng phân tích hình ảnh y tế bằng cách sử dụng mô hình như Vision Transformer để hỗ trợ chẩn đoán sơ bộ từ ảnh chụp.
- Cải thiện kiến trúc bằng cách triển khai microservices để tăng tính scalable, hoặc sử dụng mô hình edge computing để giảm độ trễ phản hồi.
- Hệ thống có thể được triển khai dưới dạng ứng dụng di động (iOS/Android) hoặc tích hợp vào các nền tảng y tế trực tuyến như bệnh viện ảo, hợp tác với các công ty dược phẩm để cung cấp tư vấn cá nhân hóa dựa trên dữ liệu người dùng.

Cuối cùng, hướng phát triển quan trọng nhất vẫn là hướng nghiên cứu dài hạn, đánh giá hiệu quả thực tế qua thử nghiệm với người dùng thực, đo lường độ chính xác và cải thiện dựa trên feedback, biến chatbot thành một công cụ y tế thông minh toàn diện hơn trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] Nguyễn Minh Hải, “Chatbot AI là gì? Ứng dụng...”, 10/01/2025 [Trực tuyến]. Địa chỉ: <https://vnptai.io/vi/blog/detail/chatbot-ai-la-gi> [Truy cập 09/09/2025]
- [2] Nguyễn Minh Hải, “LLM là gì? Khám phá ứng dụng...”, 07/01/2025 [Trực tuyến]. Địa chỉ: <https://vnptai.io/vi/blog/detail/llm-la-gi> [Truy cập 20/08/2025]
- [3] Trần Ngọc Minh, “Nhập môn về các mô hình ngôn ngữ lớn (LLMs)” [Trực tuyến]. Địa chỉ: <https://topdev.vn/blog/mo-hinh-ngon-ngu-lon-llms/#kien-truc-transformer-transformer-architecture> [Truy cập 09/09/2025]
- [4] Viettel IDC, “LLM là gì? Tìm hiểu về...”, 28/11/2024 [Trực tuyến]. Địa chỉ: <https://www.viettelidc.com.vn/tin-tuc/llm-la-gi> [Truy cập 20/08/2025]
- [5] Phuc Phan, “ChatGPT Series 5: Tìm hiểu về Retrieval Augmented Generation (RAG)”, 30/12/2023 [Trực tuyến]. Địa chỉ: <https://viblo.asia/p/chatgpt-series-5-tim-hieu-ve-retrieval-augmented-generation-rag-Ny0VGRd7LPA> [Truy cập 20/08/2025]
- [6] Ngọc Uyên, “Các phương pháp để RAG hoạt động...”, 08/07/2024 [Trực tuyến]. Địa chỉ: <https://viblo.asia/p/cac-phuong-phap-de-rag-hoat-dong-khi-len-production-3kY4g0yxJAe> [Truy cập 09/09/2025]
- [7] LamDN, “LangChain là gì?”, 24/02/2024 [Trực tuyến]. Địa chỉ: <https://viblo.asia/p/langchain-la-gi-WR5JRBPQJGv> [Truy cập 21/08/2025]
- [8] AWS, “Django là gì?” [Trực tuyến]. Địa chỉ: <https://aws.amazon.com/vi/what-is/django/> [Truy cập 21/08/2025]
- [9] Dương, “Mô hình MVT của Django” [Trực tuyến]. Địa chỉ: <https://viblo.asia/p/mo-hinh-mtv-cua-django-3kY4gdMRJAe> [Truy cập 22/08/2025]
- [10] TOS Editor, “RESTful API là gì? Cách thức...”, 16/05/2023 [Trực tuyến]. Địa chỉ: <https://www.toponseek.com/blogs/restful-api-la-gi/> [Truy cập 23/08/2025]
- [11] Hướng nội, “ReactJS là gì? Thư viện Javascript...”, 12/09/2024 [Trực tuyến]. Địa chỉ: <https://200lab.io/blog/reactjs-la-gi> [Truy cập 22/08/2025]
- [12] Miko Tech, “React Là Gì? Các Thành Phần Chính...”, 26/07/2025 [Trực tuyến]. Địa chỉ: <https://mikotech.vn/react-la-gi/> [Truy cập 23/08/2025]
- [13] Cao Lê Viết Tiến, “MySQL Workbench là gì? Hướng dẫn...”, 11/06/2025 [Trực tuyến]. Địa chỉ: <https://vietnix.vn/mysql-workbench/> [Truy cập 23/08/2025]
- [14] Phan Văn Cường, “Git và Github – Hướng dẫn...”, 27/05/2022 [Trực tuyến]. Địa chỉ: <https://unitop.com.vn/git-github.html/> [Truy cập 23/08/2025]

PHỤ LỤC